BONAFIDE CERTIFICATE

Certified that this project report “Design and Simulation of Industry Standard AMRs (AgileX Scout Mini)” is the Bonafide work of

Sailakshmi B T (2024511019) who carried out the

project work under my supervision.

SIGNATURE **SIGNATURE**

HEAD OF THE DEPARTMENT **SUPERVISOR**

TABLE OF CONTENTS:

S.No	TOPICS	PAGE
1	Introduction	5
2	Robot Specifications	5
3	Sensors Used and the Reason of Usage	6
4	SolidWorks Design	7
5	Conversion to URDF	9
6	Importing in MATLAB and Simulink	9
7	Building the Simscape Multibody Model	11
8	Creating the Warehouse	15
9	Autonomous Navigation	17
10	Wheel Slip and Traction Management	21
11	Algorithms Used	23
12	System Architecture	24
13	What Happens When the Code Runs	25
14	Additional MATLAB Visual Simulation (ISO 3691-4 Style)	30
15	How to Set It Up	34
16	Limitations and Future Enhancements	35
17	Conclusion	36

Abstract

This report describes the design and simulation of an industry-standard autonomous mobile robot (AMR), the AgileX Scout Mini four-wheel skid-steer platform, as a warehouse robot in MATLAB R2025b with Simulink and Simscape Multibody. The robot was modelled in SolidWorks (Assem2 with the parts chassis, chassis_top, W and up), exported to the Unified Robot Description Format (URDF) with STL meshes, and imported into Simulink with the Simscape Multibody function `smimport` and into the Robotics System Toolbox with `importrobot`. The imported model was turned into a working dynamic plant: the chassis is a free 6-DOF body, the four wheels are torque-driven revolute joints, and a custom tyre-ground contact model (spring-damper normal force with smoothed Coulomb friction) supplies the forces, including low-friction oil and water patches on the floor. A 3D warehouse (nine storage racks with goods, walls, pickup and delivery stations, floor boxes) is part of the multibody scene, together with four moving obstacles (two workers, a second AMR and a pallet truck). The robot is autonomous: a simulated 360° LiDAR feeds an occupancy grid, an A* planner finds and re-plans routes, a pure-pursuit controller follows them, and a mission state machine runs the sequence start, pickup, load, delivery, unload. A wheel-slip management layer (slip-ratio estimation, torque limiting, traction-control window and friction estimation) keeps the wheels from spinning or locking. In the reference run the robot completed the mission in 49.6 s over 32.0 m with 3 route corrections, 0 collisions and a minimum clearance of 0.22 m. The results are simulation only: the pose comes from the simulator (no localisation), LiDAR detection and collision checks are analytic computations on the warehouse geometry, the moving obstacles follow scripted trajectories, and no test was done on the physical robot. These limits are repeated in the relevant sections.

1. Introduction

An autonomous mobile robot (AMR) senses its surroundings, builds a representation of them, decides where to go and moves there without a fixed track or a human driver. In a warehouse an AMR carries goods between stations, and it must avoid racks, boxes, forklifts and people. Unlike an automated guided vehicle (AGV), which follows wires, tape or magnets, an AMR plans its own route and re-plans when the route is blocked. Testing such a robot on a real warehouse floor is slow and risky, so the software is first developed in simulation.

The robot used in this project is the AgileX Scout Mini, a compact four-wheel skid-steer chassis. The chassis and wheels were modelled by the author in SolidWorks (assembly Assem2) and converted to URDF. The project went through the following stages, which are also the structure of this report:

- SolidWorks CAD model of the robot, export to URDF and STL meshes (sections 4 and 5);
- import into MATLAB with `importrobot` and into Simulink with `smimport` (Simscape Multibody) (section 6);
- turning the import into a physical plant: free chassis, torque-driven wheels, tyre-ground friction (section 7);
- a 3D warehouse with racks, goods, stations, spills, static boxes and moving workers, robot and truck (section 8);
- autonomous navigation: LiDAR, occupancy grid, A* planning, re-planning, path following, mission logic (sections 9 and 11);
- wheel-slip detection and management, with a comparison of slip management on and off (section 10);
- an earlier MATLAB-script simulation of the same robot with ISO 3691-4 style safety fields (section 14);
- documentation of the code and the commands so that the work can be repeated (section 15).

The scope is simulation only. The wheel-ground forces, the wheel dynamics and the chassis motion are simulated by Simscape Multibody. The LiDAR, the obstacle geometry and the collision check are software models. Localisation error, battery and motor electrical models, the real Scout Mini firmware and hardware tests are not covered.

2. Robot Specifications

The AgileX Scout Mini is a four-wheel-drive skid-steer platform for research, education and light transport. The table lists the values of the model that was built and simulated in this project. They come from the author's SolidWorks model, the exported URDF and the

controller parameters; they are not measurements of the real robot, and the manufacturer's data sheet figures were not re-checked against them.

Parameter	Value in this project	Source
Type of drive	Four-wheel skid-steer, all wheels driven (hub-motor style torque on every wheel)	URDF, Simulink model
Links / joints	base_link + 4 wheels (w1…w4); 4 continuous wheel joints (j1…j4); fixed mounts for LiDAR, IMU, camera	`scout_mini.urdf`
Chassis mesh size	0.672 × 0.580 × 0.225 m (x, y, z in the base_link frame)	`base_link.STL` bounding box
Wheel mesh	Radius 0.090 m, width 0.124 m (largest extent of `w1.STL`); radius used for rolling in the controller and tyre model 0.085 m	w1…`w4.STL`, `scout_amr_params.m`
Track (left-right wheel distance)	0.612 m	Gazebo plug-in value in the URDF, controller
Wheel base (front-rear)	0.391 m	Joint origins in the URDF
Mass	Chassis 22.0 kg + 4 wheels × 1.5 kg + small sensor links ≈ 28.3 kg (URDF); 30 kg is used in the acceleration feed-forward	URDF, `scout_amr_params.m`
Wheel joint limits	Effort 40 N·m, velocity 25 rad/s (URDF); the controller limits motor torque to 17.5 N·m per wheel	URDF, `scout_amr_params.m`
Speed limit in the mission	$v \leq 1.2$ m/s, acceleration demand ≤ 2.5 m/s², yaw-rate demand slewed at 4 rad/s²	`scout_amr_params.m`, controller
Tyre-ground contact	Normal stiffness 12 000 N/m and damping 700 N·s/m per wheel; friction $\mu = 0.9$ on dry concrete, 0.15 on oil, 0.30 on water	`scout_amr_params.m`
Payload, battery, protection class	Not modelled	–

Applications. In this project the robot is used as a warehouse transport AMR: it drives from a start dock to a pickup station, waits while loading, drives through the aisles to a delivery station and waits while unloading. The same platform is used in research and education for navigation, mapping and control, and for indoor and outdoor patrolling.

3. Sensors Used and the Reason of Usage

An industrial AMR needs exteroceptive sensors (what is around the robot) and proprioceptive sensors (how the robot itself moves). Three groups of sensors appear in this project, and the table below states for each one whether it is part of the Simulink simulation or only declared in the robot description.

Sensor	Configuration	Reason for use	Status
2D LiDAR (Simulink simulation)	72 beams over 360° (5° spacing), maximum range 8 m, 10 Hz, exact ray–rectangle and ray–circle intersection with the warehouse geometry (`scout_wh_lidar.m`); no noise	Detects racks, boxes, workers, the second AMR and the truck; its hits build the dynamic layer of the occupancy grid, trigger re-planning and give the reactive repulsion and stop-before-contact limit	Simulated, used by the navigation

Body-state sensors (Transform Sensor blocks)	World-frame position, rotation matrix, linear and angular velocity of the base_link and of each wheel, sampled by the controller every 5 ms	Provide the robot pose and speed (stand-in for localisation and odometry) and the wheel positions for the tyre model	Simulated, ideal (true pose)
Wheel joint speed (encoder equivalent)	Joint velocity of each of the four wheel joints, 5 ms, no quantisation or noise	Closed-loop wheel-speed control and slip-ratio estimation	Simulated, ideal
Collision monitor	Ground-truth distance from the robot collision circle (r = 0.36 m) to every rack, wall, box, mover; counts overlaps	Measures safety of the run; it is not seen by the controller	Simulated (evaluation only)
2D LiDAR (URDF declaration)	Gazebo gpu_lidar: 720 samples over 360°, range 0.12–16 m, 20 Hz, Gaussian noise σ = 0.01 m	Navigation and SLAM in the earlier ROS 2 / Gazebo project that the URDF comes from	Mount imported as a body; scan model replaced by the analytic LiDAR above
IMU (URDF declaration)	Gazebo imu: 100 Hz, gyro noise σ = 0.0002 rad/s, accelerometer noise σ = 0.017 m/s²	Orientation and yaw-rate feedback for sensor fusion in ROS 2	Mount imported as a body; no IMU signal is simulated in Simulink
RGB-D camera (URDF declaration)	Gazebo rgbd_camera: 640 × 480 RGB, field of view 80°, depth 0.1–10 m	Forward perception (obstacle classification, depth)	Mount imported as a body; not simulated in Simulink
Wheel odometry (URDF declaration)	Gazebo DiffDrive plug-in: 50 Hz, wheel separation 0.612 m, wheel radius 0.08 m, limits 2.5 m/s and 3.5 rad/s	Dead-reckoning odometry in ROS 2	Not used in Simulink
Visual-simulation sensors (section 14)	LiDAR 64 beams, 8 m, σ = 0.015 m, 1 % dropouts; 6-DOF IMU with gyro bias 0.0025 rad/s; wheel odometry with 2 % drift	Illustrate sensor imperfections and ISO 3691-4 style protective fields	Simulated in the MATLAB script only

A planar LiDAR is sufficient for the warehouse of this project because every obstacle is a vertical prism (rack, box, wall, worker cylinder, truck) that extends well above the scan plane, so one plane sees everything the robot could hit. The LiDAR in the Simulink model is an analytic model, not a measurement: it has no noise, no dropped returns and no minimum range. The robot pose used by the navigation is the true simulator pose, so no localisation, SLAM or sensor fusion is implemented in the Simulink project; the EKF, SLAM Toolbox and Nav2 configuration of the ROS 2 repository the URDF comes from were not run in this work.

4. SolidWorks Design

The robot was modelled by the author in SolidWorks. The files are in the repository Saibts/AgileX-scout-mini-sw (uploaded on 17 August 2026): the assembly `Assem2.SLDASM` and the parts `chassis.SLDPRT`, `chassis_top.SLDPRT`, `W.SLDPRT` (wheel) and `up.SLDPRT`. The assembly consists of one chassis body with a raised top cover carrying two rails, four wheels with hub spokes and the lettering "AGILE X" and "SCOUT MN" on the sides. The wheel axles are carried by four small brackets on the chassis sides. The URDF-export coordinate systems and axes (Origin_j1…, Axis_j1…) for the four wheel joints were

created in the assembly for the exporter. Native SolidWorks files cannot be opened in MATLAB, so the geometry reaches MATLAB through the exported STL meshes (section 5).

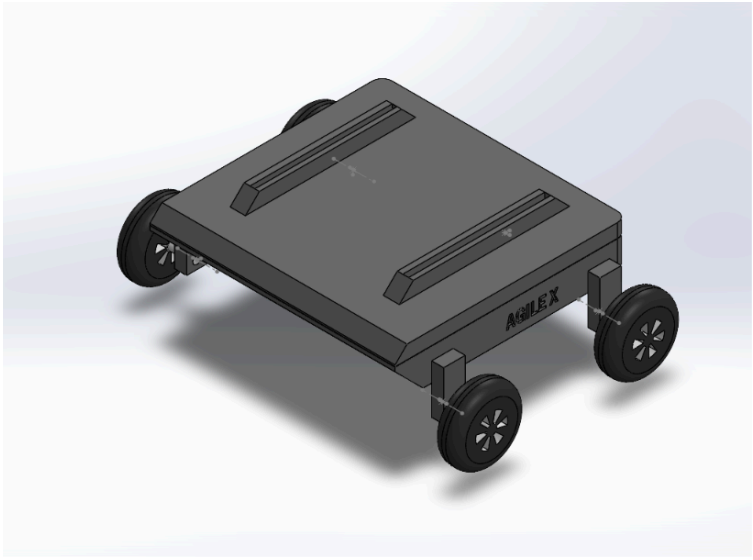


Fig. 1: Isometric view of the Scout Mini model in SolidWorks

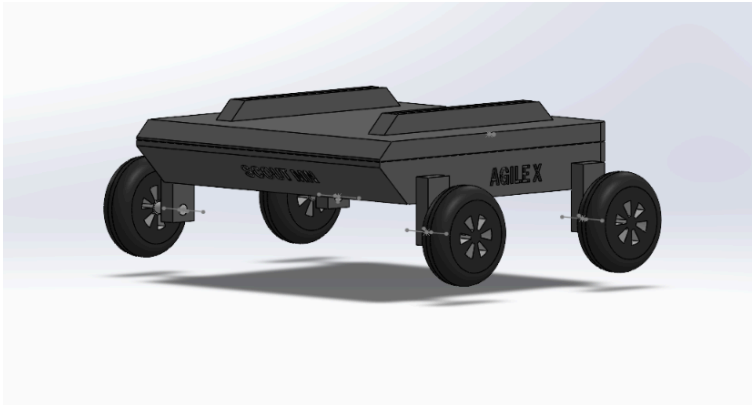


Fig. 2: Side view with the "AGILE X" and "SCOUT MN" lettering

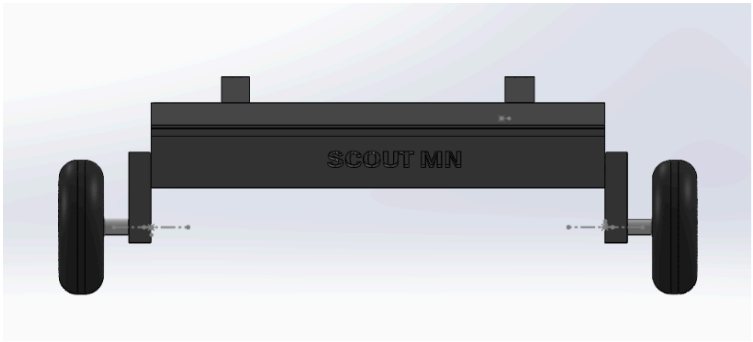


Fig. 3: Front view showing the wheel axles and brackets

The mass properties and the dimensions that are used by the simulation are the ones stored in the URDF and the STL files (section 5). How closely the CAD model matches the real Scout Mini was not verified against the real robot.

5. Conversion to URDF

The Unified Robot Description Format (URDF) is an XML file that describes a robot as a tree of links (rigid parts with mass, inertia, visual and collision shapes) and joints (how a child link is attached to its parent: type, origin and axis). An STL file holds only the triangle mesh of a shape; the URDF says where each mesh sits and how it moves. The meshes are in metres.

The assembly was exported with the SolidWorks-to-URDF exporter into the ROS 2 package assem2_robot of the author's repository AgileX_ScoutMini_JAZZY (urdf/`Assem2.SLDASM`.urdf and `meshes/base_link.STL`, `w1.STL`…`w4.STL`). That URDF was later edited by hand for the ROS 2 / Gazebo project: it carries analytic collision shapes (a box for the chassis, cylinders for the wheels), an adjusted low centre of mass, fixed mounts for the LiDAR, IMU and camera, and Gazebo plug-ins and sensor definitions. For the MATLAB work the file was copied to `scout_mini.urdf`, and the only change was to replace the ROS path `package://assem2_robot/meshes/` by the relative path `meshes/`, because MATLAB does not know the package:// convention.

Quantity	Value	How it was obtained
Links / joints	6 main links (base_footprint, base_link, w1…w4) plus the sensor links; 4 continuous wheel joints	`scout_mini.urdf`
Link masses	base_link 22.0 kg, wheels 1.5 kg each, LiDAR mount 0.08 kg + LiDAR 0.125 kg, IMU 0.01 kg, camera 0.04 + 0.07 kg	URDF inertial tags
Chassis inertia (about its centre of mass)	Ixx 0.35, Iyy 0.45, Izz 0.65 kg·m²	URDF
Wheel inertia	Ixx 0.005, Iyy = Izz 0.003 kg·m²	URDF
Wheel joint axes	Local x-axis (+1 for j1, j2; −1 for j3, j4); joint limit effort 40 N·m, velocity 25 rad/s	URDF
Wheel joint positions in base_link	(0.139, −0.297), (0.139, 0.094), (−0.473, −0.297), (−0.473, 0.093) m; z ≈ −0.146 m	URDF joint origins
Base_link in base_footprint	(−0.102, 0.167, 0.227) m, yaw +90°	URDF
Chassis mesh bounding box	x −0.503…0.169, y −0.415…0.165, z −0.169…0.056 m	`base_link.STL`
Wheel mesh	radius 0.090 m, width 0.124 m	`w1.STL`
Mesh triangles	base_link 1 628, each wheel 2 152	STL headers

The URDF contains base_footprint as a root link joined to base_link by a fixed joint. This fixed joint is the one that was replaced by a 6-DOF joint in the Simulink model (section 7).

6. Importing in MATLAB and Simulink

The robot was imported in two ways. First, `importrobot` (Robotics System Toolbox) turns the URDF into a `rigidBodyTree`, which is used for visualisation and to read the wheel transforms. Second, `smimport` (Simscape Multibody) turns the same URDF into a Simulink model made of Simscape Multibody blocks: Solid blocks with the STL geometry, Rigid Transforms for the joint frames, Revolute Joints for the wheels and a Weld Joint for the base footprint.

6.1 Code and commands

```
>> cd 'D:\academic_shit\SEM5-III\AMR ROS\simulink_amr'
>> rbt = importrobot('scout_mini.urdf');       % rigidBodyTree
>> show(rbt, homeConfiguration(rbt), 'Visuals', 'on');
>> smimport('scout_mini.urdf');                % Simscape Multibody model
(several minutes)
>> build_scout_amr_model                       % imports (or reuses the
cached import), then builds scout_amr_sim.slx
```

`smimport` takes several minutes for this model, so build_scout_amr_model saves the raw import as `scout_import_raw.slx` and reuses it on later builds. An important property of the imported model: the joint frames created by `smimport` have their Z axis along the wheel axle (not the X axis written in the URDF). The tyre model and the slip-ratio sign convention were built on this fact; using the wrong axis had put the wheel hubs up to 10 cm off and tilted the chassis at the start of the simulation.

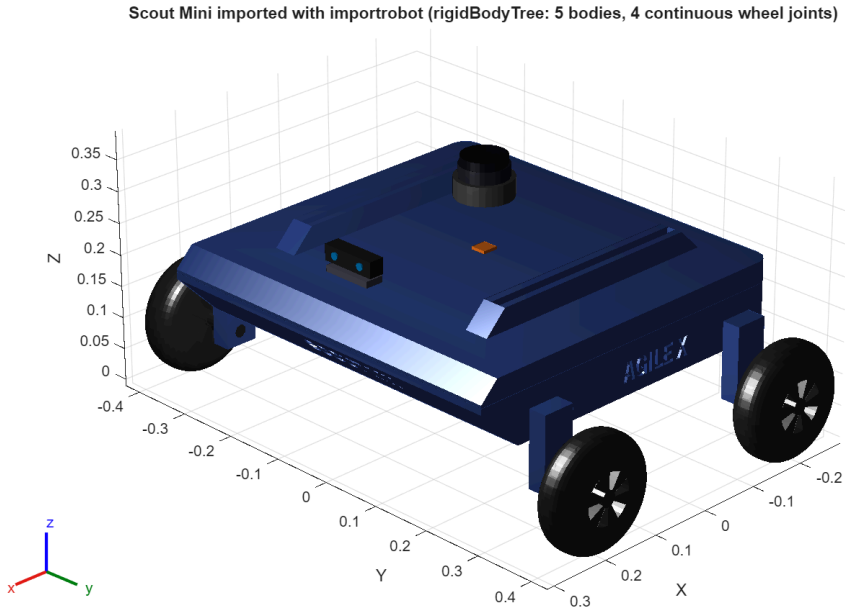


Fig. 4: The Scout Mini imported with importrobot (rigidBodyTree)

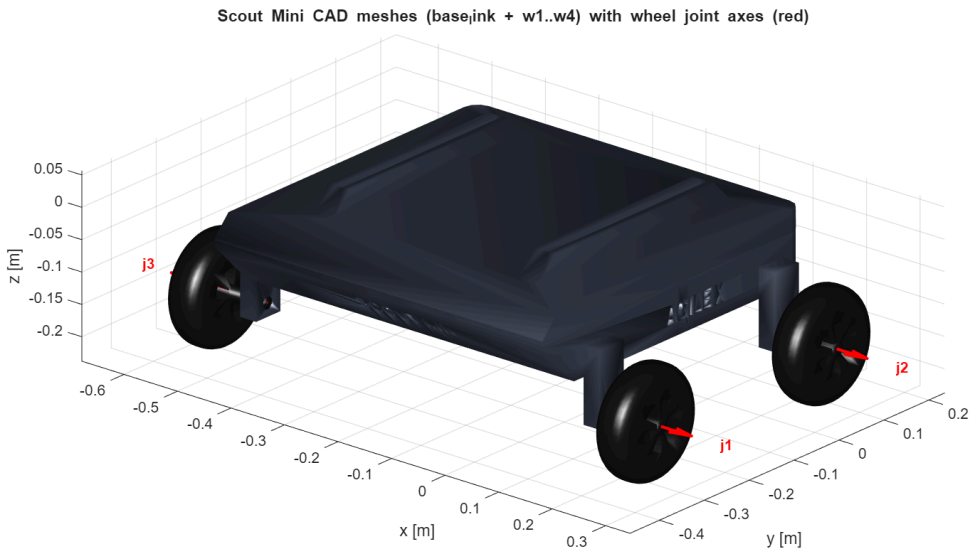


Fig. 5: The CAD meshes (base_link and w1…w4) placed with the URDF transforms; red arrows are the wheel joint axes

7. Building the Simscape Multibody Model

The raw import is a rigid, welded robot. `build_scout_amr_model.m` turns it into a dynamic plant by the following steps, which are executed in the order given and saved as `scout_amr_sim.slx`.

- The Weld Joint between base_footprint and base_link is replaced by a 6-DOF Joint, so the chassis can translate and rotate freely in the world.
- The four wheel revolute joints (j1…j4) get TorqueActuationMode = InputTorque (the controller applies a torque to every wheel) and SenseVelocity = on.
- Transform Sensor blocks measure the world-frame pose and velocity of the chassis and of each wheel (position, velocity, rotation, angular velocity), and Simulink-PS / PS-Simulink converters connect them to the signal domain.
- Four External Force and Torque blocks, one per wheel, apply the tyre forces computed by the tyre-ground model.
- The controller and the tyre model are MATLAB Function blocks that call `scout_amr_controller.m` and `scout_amr_tires.m`. They run at the control period of 5 ms through zero-order-hold blocks.
- The 3D warehouse and the moving obstacles are added as Brick and Cylinder Solid blocks (section 8).
- Logging blocks store the controller, tyre, LiDAR, path and mover signals (To Workspace).
- Solver: `ode23t` (variable step, maximum step 0.01 s), stop time 120 s; the simulation stops itself when the mission is done.

7.1 Tyre-ground contact model

The Scout Mini wheels roll on the floor, so the multibody model needs forces between the wheels and the ground. Simscape Multibody has no built-in friction contact for this case, therefore a contact model was written (`scout_amr_tires.m`) and is evaluated once for every wheel:

- Contact point: below the wheel hub on the axis perpendicular to the axle, at the rolling radius r = 0.085 m. The hub position follows from the wheel frame position plus 0.062 m along the axle.
- Normal force: $F_z = k_c \cdot \delta + c_c \cdot \dot{\delta}$ (never negative) with k_c = 12 000 N/m, c_c = 700 N·s/m per wheel; δ is the penetration of the tyre into the floor.
- Friction: the contact-point velocity gives a sliding velocity; the tangential force is $\mu \cdot F_z$ scaled by a smoothed sign function with smoothing velocity v_0 = 0.03 m/s (smoothed Coulomb friction).
- μ comes from a floor map: μ = 0.9 on dry concrete, 0.15 on the oily loading dock at the start, and 0.30 on two water spills in the aisles.
- Application: the force is resolved in the wheel frame and applied at the wheel-frame origin through the External Force and Torque block, together with the moment of the force about the offset contact point.
- Diagnostics for each wheel: Fz, longitudinal force, lateral force, slip ratio and μ (11 outputs per wheel in the log).

With 30 kg of robot mass the static load per wheel is about 74 N, which compresses the contact by about 6 mm; this agrees with the normal-load trace in Fig. 15 (60–80 N when driving straight).

The model is organised so that its top level stays small. It holds only the World Frame, the Mechanism Configuration, the Solver Configuration and four colour-coded subsystems (Fig. 6); all signals between the subsystems travel through Goto / From tags, so the top level contains no signal wires, only the physical frame connections of the multibody tree.

The subsystem Scout_Mini_Robot holds the imported robot (chassis, wheels, sensor mounts), the sensors, the wheel drives and the nested subsystem Tyre_Ground_Contact_Model with one MATLAB Function block per wheel. The `tc_on` switch is at scout_amr_sim/Mission_and_Slip_Controller/`tc_on`. The blocks of each subsystem are listed in the table below.

Subsystem	Blocks	Role
Scout_Mini_Robot	base_footprint, chassis_6dof, base_link, w1…w4, j1…j4, lidar / imu / camera mounts, wheel Transform Sensors, tau1…tau4 drives, WheelWrench1…4 (External Force and Torque), Tyre_Ground_Contact_Model	Imported robot with free chassis, driven wheels, sensors and tyre forces
Warehouse	Floor, 3 low-μ patches, 3 pads, 9 racks (+ 27 goods blocks), 4 walls, 2 floor boxes	3D static environment
Moving_Obstacles	Worker_1, Worker_2, Second_AMR, Pallet_Truck with two prismatic joints each, mover demux	Prescribed-motion obstacles
Mission_and_Slip_Controller	`tc_on`, zero-order holds, Controller (MATLAB Function), logging blocks	Autonomy, slip management, logging

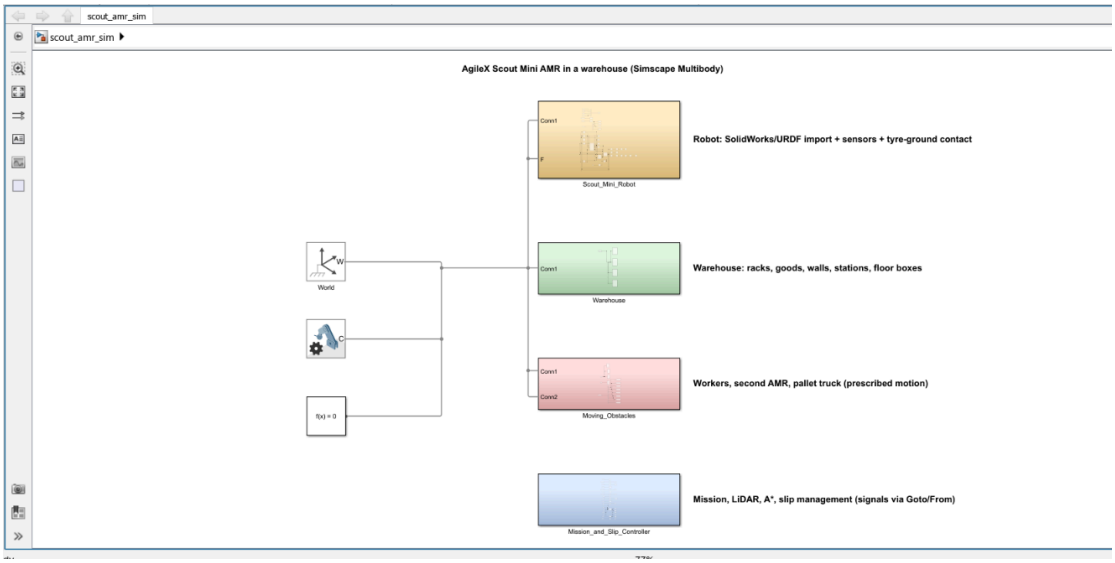


Fig. 6: Top level of scout_amr_sim.slx in Simulink: World, Mechanism Configuration, Solver Configuration and the four subsystems Scout_Mini_Robot, Warehouse, Moving_Obstacles and Mission_and_Slip_Controller

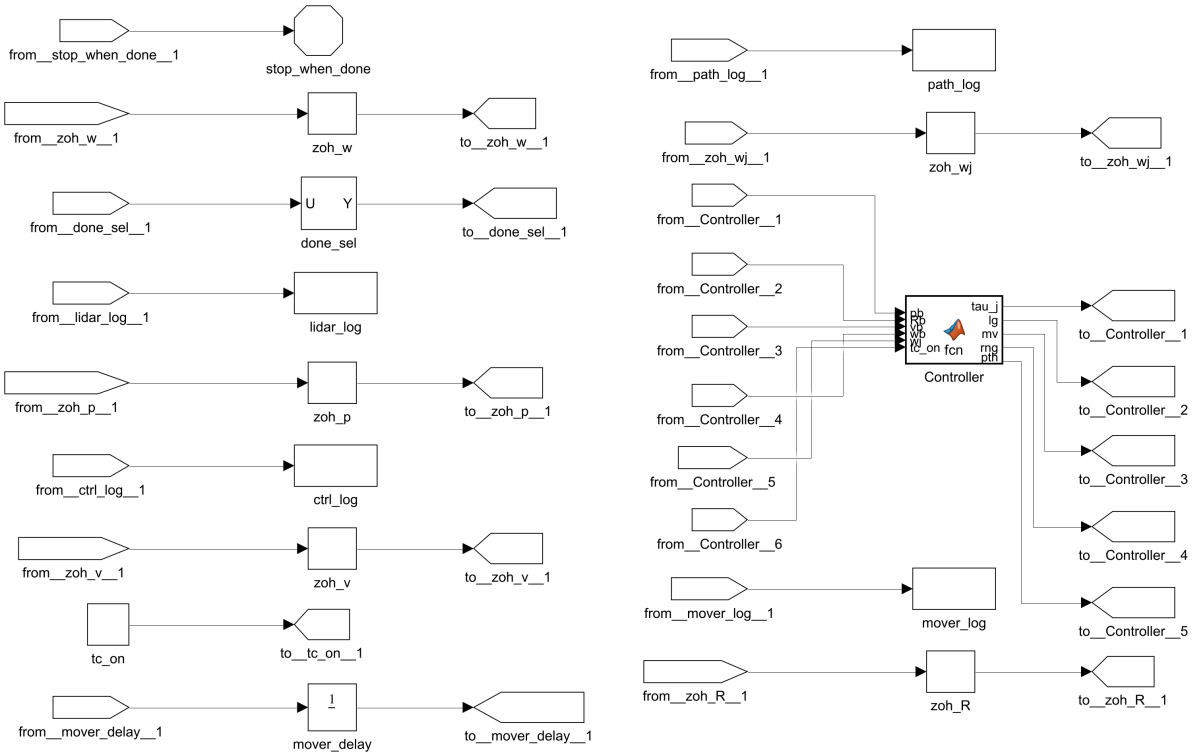


Fig. 7: Subsystem Mission_and_Slip_Controller (zero-order holds, controller function, logging; tc_on selects slip management). The subsystem is shown in two halves: left half first, then right half

Every signal enters the subsystem through a From block and leaves through a Goto block. The zero-order holds (zoh_p, zoh_v, zoh_w, zoh_R, zoh_wj) sample the chassis position, velocity, wheel speeds, rotation and joint speeds at the control period of 5 ms. The Controller block is a MATLAB Function block that calls `scout_amr_controller.m`: its inputs are the sampled states and the constant `tc_on` (1 = slip management on, 0 = off), its outputs are the four wheel torques (tau_j), the 44-value log vector (lg), the mover positions (mv), the LiDAR ranges (rng) and the planned path (pth). The logging blocks (ctrl_log, lidar_log, path_log, mover_log) store these signals for the figures, mover_delay is the Unit Delay that removes the algebraic loop, done_sel selects the mission-done flag, and stop_when_done ends the simulation when the mission is complete.

8. Creating the Warehouse

The warehouse is created twice from the same parameters (`scout_amr_params.m`): as Simscape solids, so that it is part of the 3D multibody scene, and as geometry for the LiDAR, the planner and the collision monitor (`scout_wh_world.m`). The layout is a 31 m × 14.2 m hall with three rack rows, three aisles, two cross-aisles and open docks at both ends. The robot starts at the origin on the dock, facing +x. All dimensions were chosen by the author for the demonstration.

Element	Layout
Hall	x −2.5…28.5 m, y −2.2…12 m, walls 1.2 m high
Rack rows	Row A y 1.5…2.7 m, row B y 5.7…6.9 m, row C y 9.9…11.1 m; each row in three segments at x 3…8.6, 10.2…15.8 and 17.4…23 m (2.4 m high frames with 3 shelves of goods)
Aisles	Aisle 1 y −1.5…1.5 m (service lane), aisle 2 y 2.7…5.7 m, aisle 3 y 6.9…9.9 m; cross-aisles at x 8.6…10.2 and 15.8…17.4 m
Stations	Start pad (0, 0); pickup station (1.2, 4.2) m at the dock end of aisle 2; delivery station (26, 8.4) m at the dock end of aisle 3; loading and unloading time 3 s each
Static obstacles not in the planner's map	Floor box 0.8 × 0.8 m in aisle 2 at (6.5, 3.9) m; floor box in aisle 3 at (13, 8.9) m
Low-friction patches	Oil, μ = 0.15: x −1.5…2.5, y −1.5…1.5 (start dock). Water, μ = 0.30: x 9…11, y 2.7…5.7 (aisle 2) and x 15.5…18, y 6.9…9.9 (aisle 3)
Moving obstacle 1	Worker 1 (cylinder d = 0.6 m) repeatedly crossing aisle 2 at x = 14 m, pausing at the shelves and waiting while the AMR is close
Moving obstacle 2	Worker 2 walking along aisle 3 at y = 7.5 m (x = 15.5…24.5 m), stops when the AMR is within 2.2 m
Moving obstacle 3	Second AMR (0.9 × 0.6 m) shuttling along aisle 1 at y = −0.4 m
Moving obstacle 4	Pallet truck (1.4 × 2.6 m) parked at the east dock; released when the robot passes x = 12 m, it drives into aisle 2 at 1 m/s and stops at x = 20.4 m, blocking the aisle

The moving bodies are Simscape solids on two prismatic joints each (x and y). The positions come from the world model through motion-input joints (filtered, second order), so they move inside the multibody scene. A Unit Delay in the feedback path removes the algebraic loop between the controller and these joints.

8.1 Planning map

The planner works on an occupancy grid with 0.2 m cells (155 × 71 cells). The racks and walls are known in advance (static layer). Boxes, workers, the second AMR and the truck are not in the prior map; they enter the grid only when the LiDAR sees them (dynamic layer, held for 2 s). All occupied cells are inflated by 0.48 m (robot collision radius 0.36 m plus a margin).

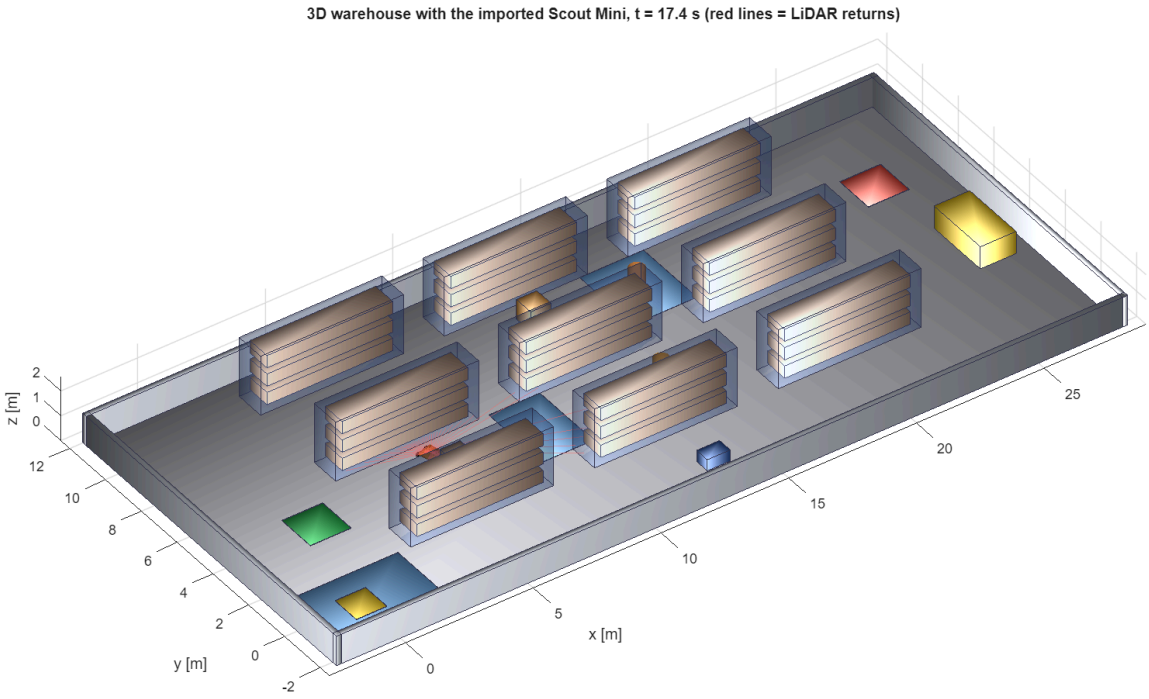


Fig. 8: 3D view of the warehouse with the Scout Mini avoiding the floor box (t = 17.4 s).

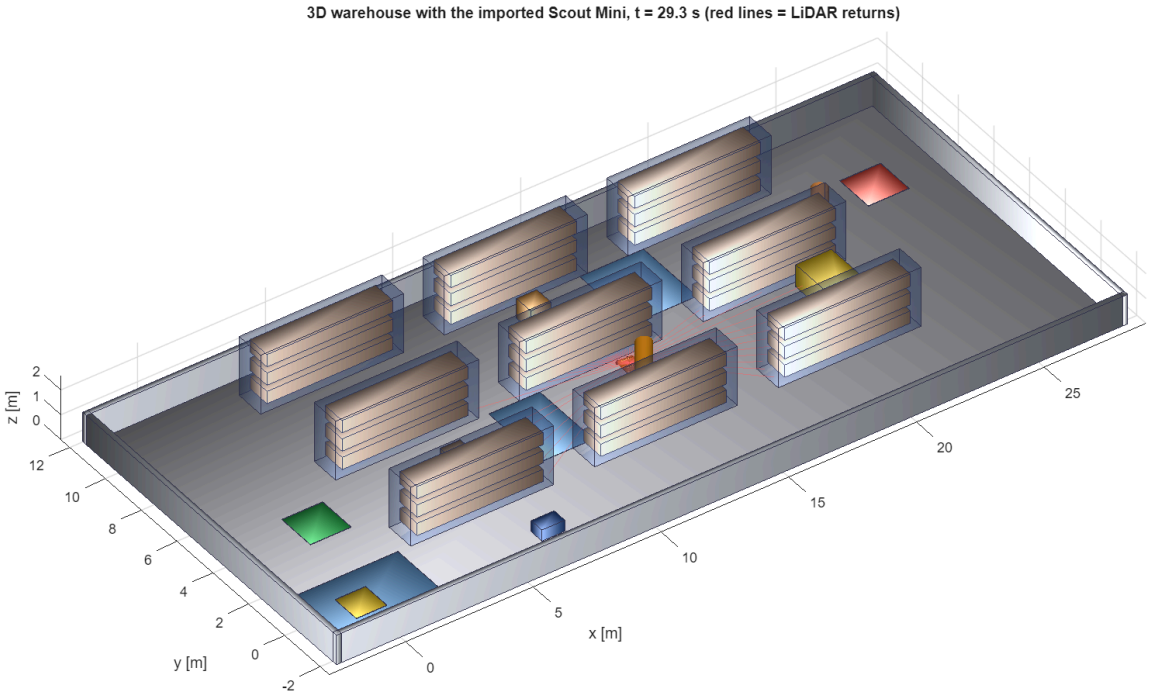


Fig. 9: 3D view while a worker crosses the aisle and the truck blocks aisle 2 (t = 29.3 s); red lines are the LiDAR returns

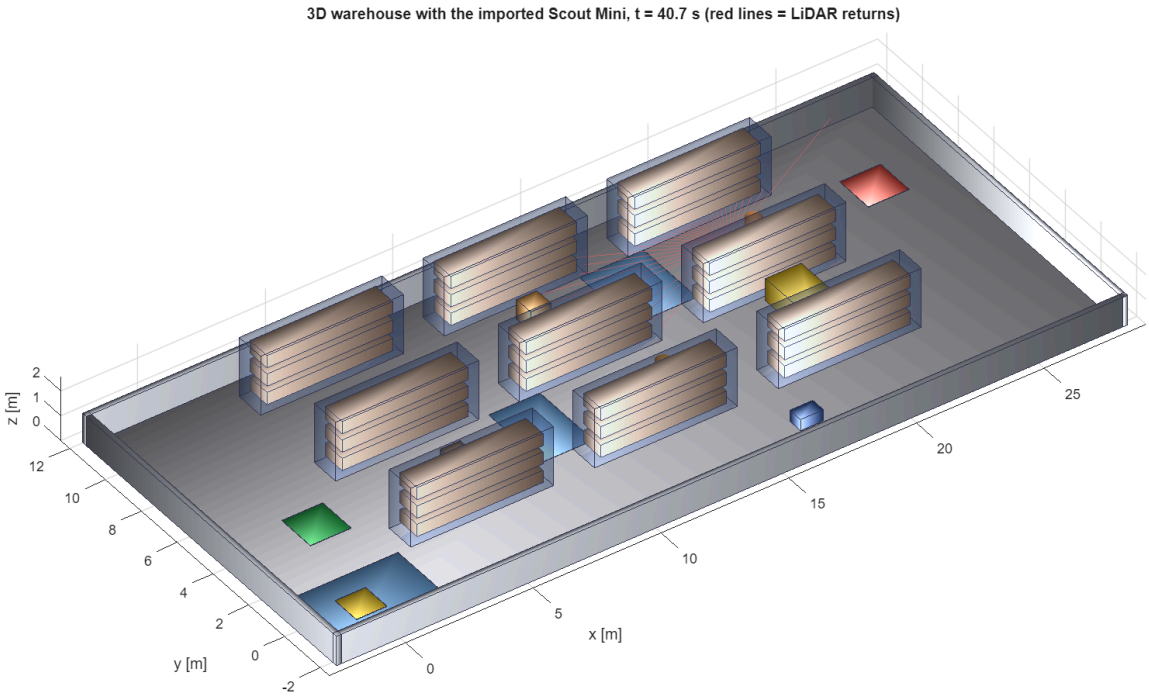


Fig. 10: 3D view of the final approach to the delivery station through aisle 3 (t = 40.7 s)

9. Autonomous Navigation

Autonomous navigation is part of the Simulink model: the MATLAB Function block Controller calls `scout_amr_controller.m` every 5 ms, which calls `scout_wh_nav.m` for the perception, planning and mission logic (Fig. 7). The simulation is started with `run_scout_warehouse_simulation.m`, which also produces all plots.

9.1 Commands

```
>> cd 'D:\academic_shit\SEM5-III\AMR ROS\simulink_amr'
>> open_system('scout_amr_sim')                    % Simulink model; press
Run to see the Mechanics Explorer 3D view
>> run_scout_warehouse_simulation                  % simulation with 3D
window, then all graphs
>> run_scout_warehouse_simulation(false)           % headless
>> run_scout_warehouse_simulation(false, true)     % slip management ON and
OFF, with comparison plot
>> run_scout_warehouse_simulation('replot')        % redraw the figures from
the saved results
```

Console output of the reference run (slip management on):

```
========== WAREHOUSE MISSION RESULTS (Slip management ON) ==========
mission time:        49.6 s (done flag)
start -> pickup:     reached at t = 6.1 s
pickup -> delivery:  left at t = 9.1 s, delivered at t = 46.6 s
distance driven:     32.0 m   mean speed 0.64 m/s   top speed 1.20 m/s
route corrections:   3     collisions: 0     min clearance: 0.22 m
closest LiDAR hit:   0.58 m   peak slip ratio: 0.48   wheel energy: 2670 J
```

9.2 One mission from start to delivery

The mission has five states: drive to the pickup station, load for 3 s, drive to the delivery station, unload for 3 s, done. The robot reached the pickup station at 6.1 s, left it at 9.1 s and stopped at the delivery station at 46.6 s. The run ended at 49.6 s. The simulation was repeated with the same settings and gave the same numbers (the world, the workers and the controller are deterministic, there is no sensor noise).

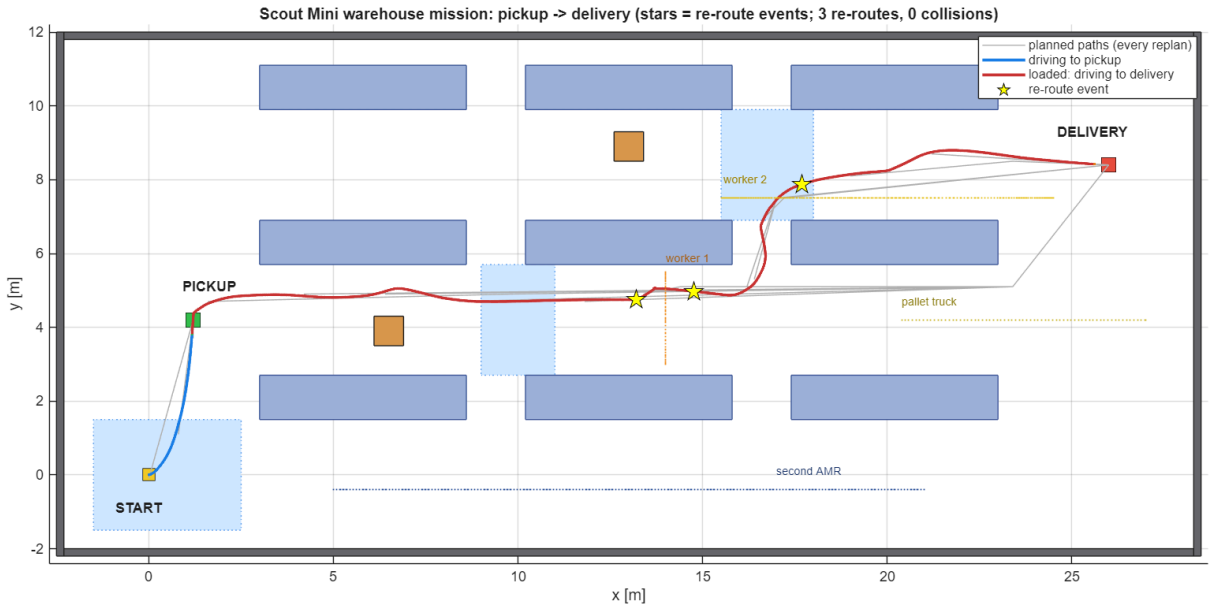


Fig. 11: Warehouse map with the driven trajectory (blue: to pickup, red: to delivery), all planned paths (grey) and the re-route events (stars); dotted lines are the paths of the moving obstacles

The six demonstration scenarios happen in one mission. The table below gives the time of each one in the reference run.

No.	Scenario	Time in the run	What happens
1	Aisle traversal	9–46 s	The robot drives along aisle 2 and then aisle 3 between the racks with about 0.2–0.6 m of clearance
2	Navigating around shelves	≈30–38 s	The robot leaves aisle 2 through the cross-aisle at x ≈ 16.5 m and continues in aisle 3
3	Avoiding a floor box	≈17 s	The LiDAR detects the box in aisle 2; the planner adds it to the map and the path bends around it
4	Avoiding a moving human	≈29 s	Worker 1 crosses aisle 2; the robot slows down and waits or steers around until the lane is clear
5	Pickup to delivery	9.1–46.6 s	Loaded drive from the pickup station to the delivery station
6	Alternative path when blocked	25.1, 30.2, 37.6 s	The pallet truck blocks aisle 2; the robot detects the blockage, backs away, re-plans through the cross-aisle and aisle 3, where worker 2 is also avoided

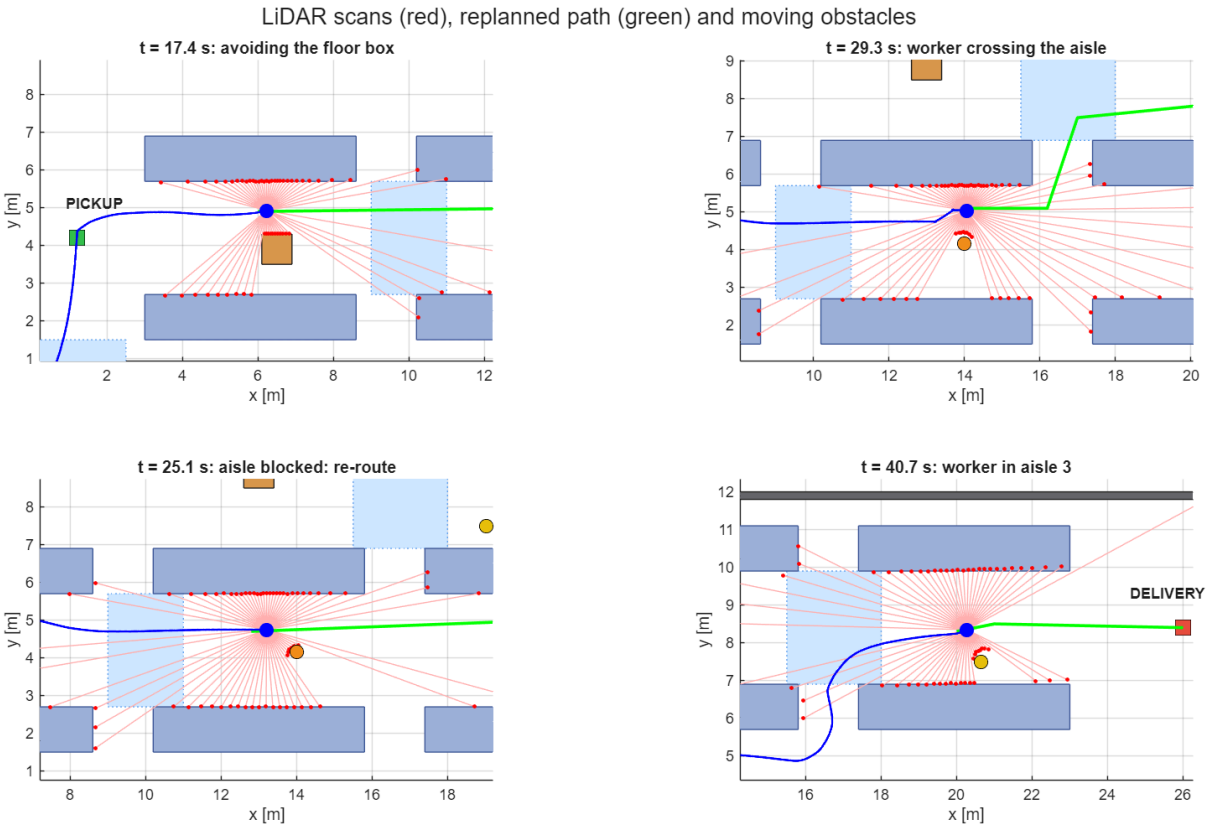


Fig. 12: LiDAR scans (red), re-planned path (green), driven path (blue) and moving obstacles at four moments of the mission

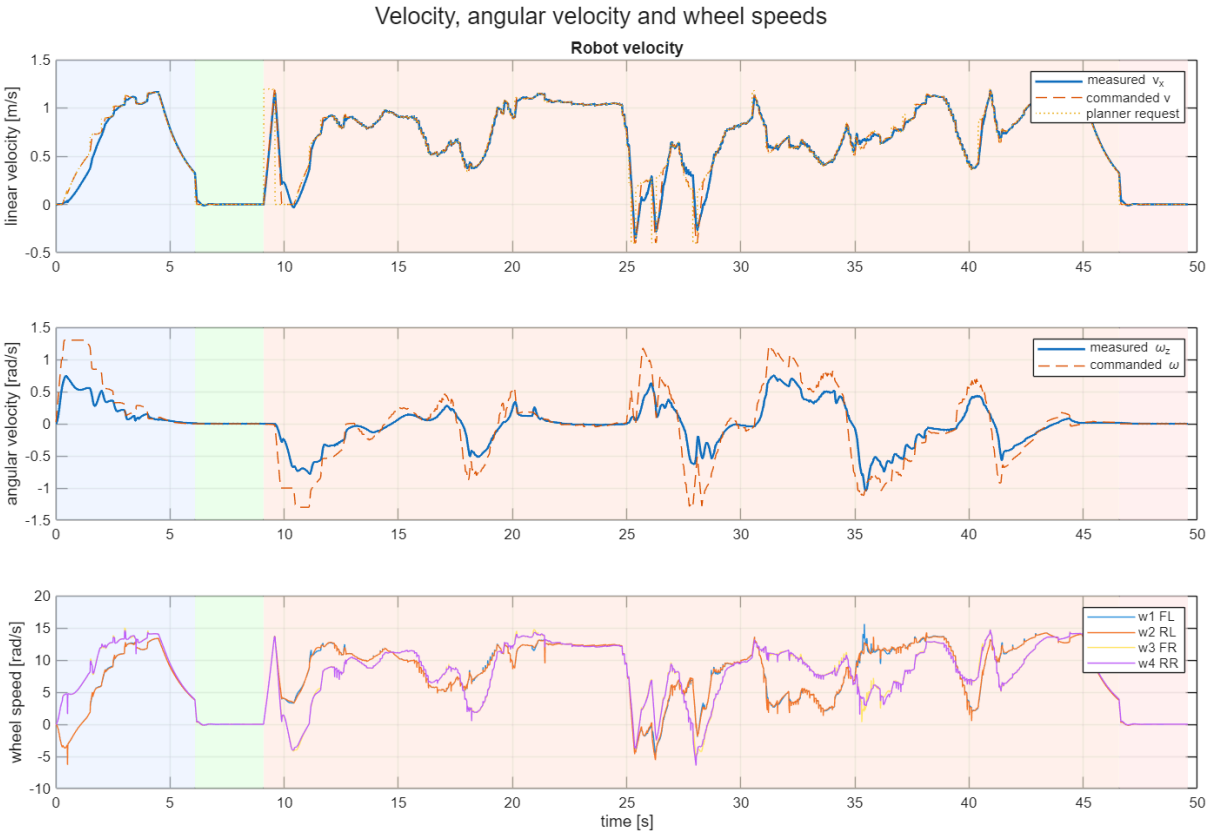


Fig. 13: Linear and angular velocity (commanded and measured) and the four wheel speeds

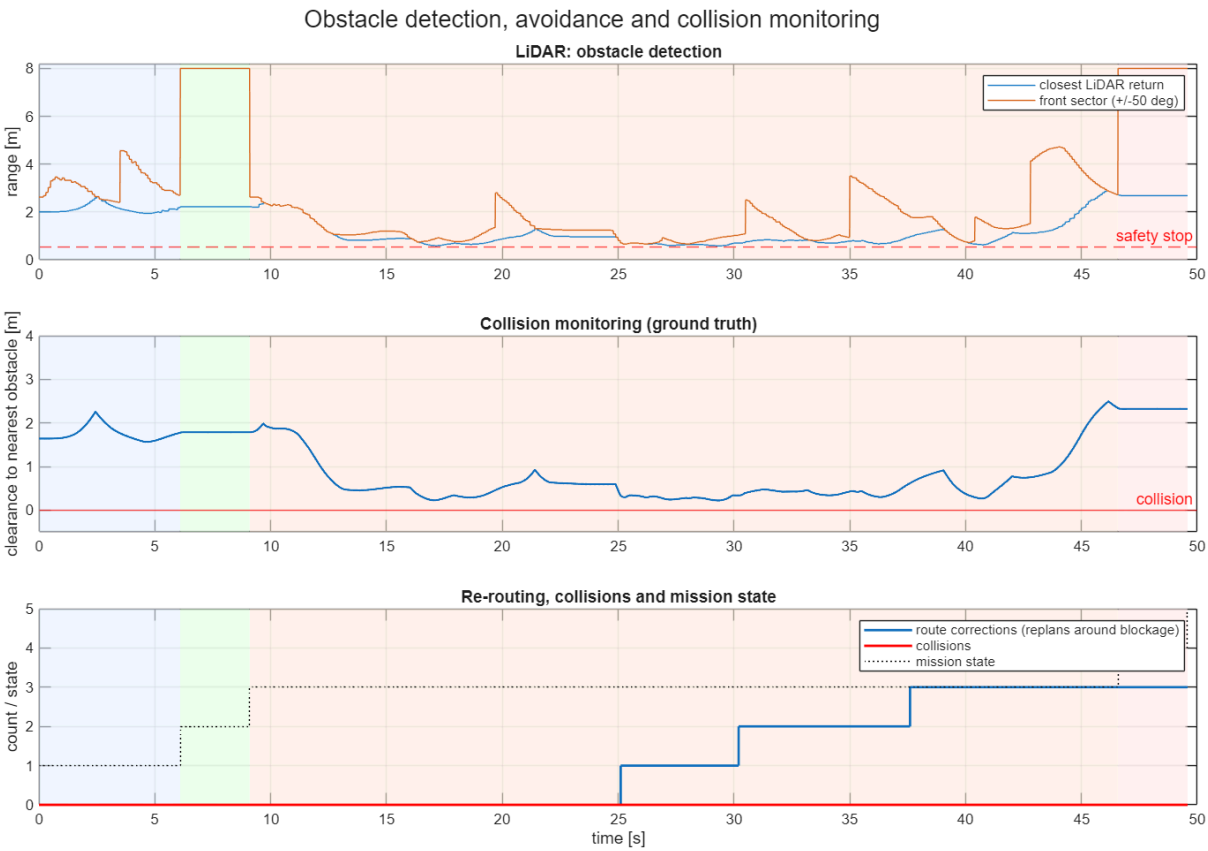


Fig. 14: LiDAR ranges, ground-truth clearance, route corrections and mission state (shaded: drive to pickup, load, drive to delivery, unload)

Quantity	Value
Mission completed	yes
Mission time	49.6 s
Distance driven	32.0 m
Mean / top speed	0.64 m/s / 1.20 m/s
Route corrections (replans around a blockage)	3
Collisions (collision circle r = 0.36 m overlaps an obstacle)	0
Minimum clearance to any obstacle	0.22 m
Closest LiDAR return	0.58 m
Peak wheel slip ratio / RMS slip ratio	0.48 / 0.06
Peak yaw rate	1.04 rad/s
Wheel mechanical work ($\Sigma\lvert\tau\cdot\omega\rvert$)	2670 J
RMS wheel torque / peak wheel torque	1.97 N·m / 6.17 N·m
Wall-clock time of the simulation run	about 30 s on the author's computer (headless)

9.3 Limits of these results

- Simulation only; no run on the physical robot. The pose is the true pose of the simulator (no localisation or odometry error).
- One layout and one mission. The simulation is deterministic, so repeating it gives the same numbers; the result shows that the chain works, not that it works on every

layout.

- The workers are partly cooperative: they pause when the AMR is close. They are still obstacles that the robot must detect and avoid, but they are gentler than real traffic.
- The LiDAR and the collision check are analytic computations on the warehouse geometry. The robot does not collide physically with the racks or the movers: there is no Simscape contact between the robot and the obstacles, and the movers cannot be pushed.
- The minimum clearance of 0.22 m is measured from the robot's collision circle (r = 0.36 m), which is a simplification of the real chassis outline.

10. Wheel Slip and Traction Management

The floor of the warehouse is not uniform, so the wheels can lose grip. The simulation includes three friction zones (dry concrete, an oily loading dock at the start and water spills in two aisles), and a skid-steer robot also scrubs its wheels sideways in every turn. The controller therefore monitors the slip of every wheel and manages it.

10.1 How slip is detected and managed

- Slip ratio of each wheel: $\kappa = (r\cdot\omega - v_k) / \max(|r\cdot\omega|, |v_k|, 0.8\ \text{m/s})$. Here ω is the wheel speed, r the radius and v_k the ground speed at the wheel's lateral position (chassis speed minus yaw rate times the wheel offset). The 0.8 m/s floor avoids division by zero at standstill.
- Detection: a wheel slips when $|\kappa| > 0.12$; a slip timer records how long this lasts.
- Traction-control window: the wheel-speed target is limited to the speed that matches the ground ± a tolerance (anti-spin on acceleration, anti-lock on braking). The allowed slip is regulated to about 0.08.
- Torque limiting: above the threshold the available torque is reduced linearly, down to 10 % of the 17.5 N·m limit at $\kappa = 0.40$.
- Friction estimation: while slip is detected the friction estimate $\hat{\mu}$ decreases by 0.5 per second (not below 0.10); when there is no slip it recovers at 0.15 per second up to 0.9.
- Friction-aware acceleration: the acceleration demand is limited to $0.85\cdot\hat{\mu}\cdot g$, so the robot accelerates gently on a slippery floor.
- Wheel-speed control: a PI controller (Kp = 0.3, Ki = 8) with an acceleration feed-forward term (30 kg · a · r / 4) drives each wheel to its target; the integrator is clamped to the current torque limit.

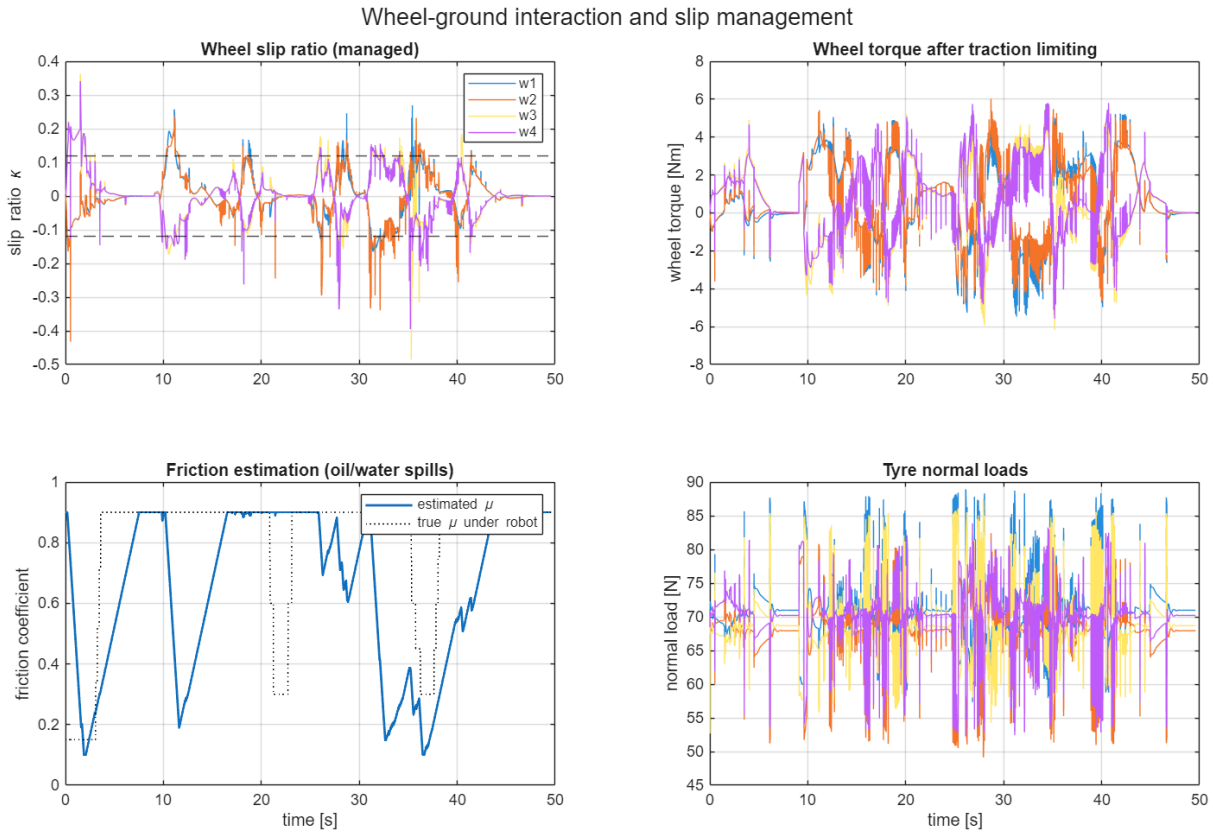


Fig. 15: Wheel slip ratios with the ±0.12 threshold, wheel torques after limiting, friction estimate against the true friction under the robot, and tyre normal loads

The friction estimate falls as low as 0.10 in the low-friction zones and recovers on dry floor, following the true friction with a lag of several seconds (Fig. 15, bottom left). The slip ratio stays mostly within ±0.2 with short spikes during sharp turns, reversing and in the spills. The torque and normal-load traces are noisy because the contact model is stiff and the controller works at 5 ms; this chatter was not tuned away.

10.2 Slip management on and off

The mission was simulated twice with the same world: with the slip management of section 10.1 (`tc_on` = 1) and with a plain wheel-speed controller (`tc_on` = 0), which has no traction window, no torque limiting and no friction-aware acceleration limit.

Quantity	Slip management ON	Slip management OFF
Mission completed	yes	yes
Mission time [s]	49.6	45.5
Distance driven [m]	32.0	31.5
Peak slip ratio (worst wheel)	0.48	0.64
RMS slip ratio (all wheels)	0.064	0.070
Share of time with worst-wheel $\|\kappa\| > 0.2$	2.1 %	5.4 %
Wheel mechanical work [J]	2670	2633
Collisions	0	0
Minimum clearance [m]	0.22	0.19

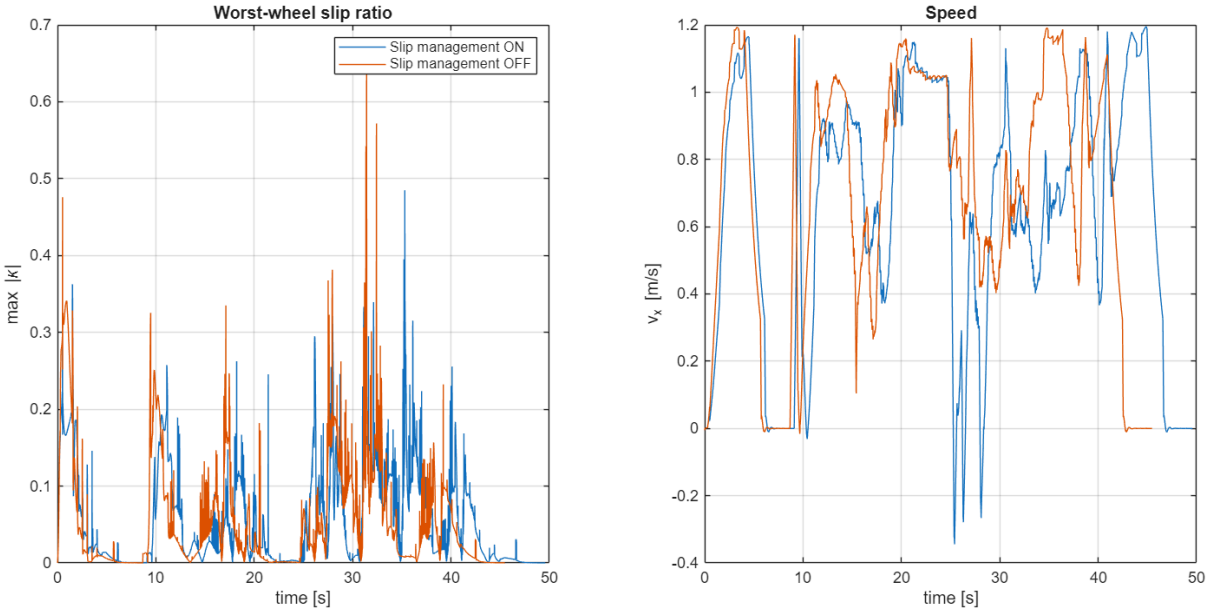


Fig. 16: Worst-wheel slip ratio and forward speed with slip management on (blue) and off (orange)

With slip management the peak slip ratio is lower (0.48 against 0.64) and the worst wheel spends less time above a slip ratio of 0.2 (2.1 % against 5.4 % of the time). The price is time: the friction-aware acceleration limit and the torque limiting make the robot slower, so the mission takes 49.6 s instead of 45.5 s. The energy and the RMS slip ratio are almost the same. In this layout the plain controller also finished the mission without a collision, so the benefit of the slip management is smoother, more controlled wheel-ground behaviour, not the difference between success and failure. A single run per case on one layout does not allow a stronger statement.

11. Algorithms Used

The navigation and control stack is written in MATLAB and runs inside the Simulink model. SLAM, AMCL, Nav2 planners and behaviour trees were NOT used in the Simulink project; their roles are filled by the occupancy grid and A* (global planning), pure pursuit with reactive repulsion (local control) and a simple mission state machine.

Algorithm	File	What it does	Main parameters
Analytic LiDAR	`scout_wh_lidar.m`	Casts rays from the robot against rectangles (racks, walls, boxes, AMR, truck) and circles (workers) and returns the first hit	72 beams, 8 m, 10 Hz, no noise
Occupancy grid with inflation	`scout_wh_nav.m`	Static map plus a dynamic layer built from LiDAR hits; occupied cells are dilated by the robot radius plus a margin	0.2 m cells, 155 × 71, inflation 0.48 m, dynamic hold 2 s
A* global planner	`scout_wh_plan.m`	8-connected A* with Euclidean heuristic and a binary heap; the path is smoothed by line-of-sight shortcuts and limited to 80 points	replan every 0.5 s or when the path is blocked
Pure pursuit	`scout_wh_nav.m`	Steers towards a point on the path ahead of the robot	look-ahead 0.9 m, v ≤ 1.2 m/s

Reactive repulsion and stop-before-contact	`scout_wh_nav.m`	Pushes the robot away from close LiDAR hits and caps the speed at $v = \sqrt{(2 \cdot 0.8 \cdot (\text{range} - 0.55))}$ in front of obstacles	braking 0.8 m/s², stop margin 0.55 m
Retreat behaviour	`scout_wh_nav.m`	If an obstacle closes in and the rear is free, the robot backs away and re-plans	stuck detection, 4 s re-route debounce
Mission state machine	`scout_wh_nav.m`	to pickup → load → to delivery → unload → done	dwell 3 s, goal radius 0.40 m
Skid-steer kinematics	`scout_amr_controller.m`	$\omega_{left} = (v - \tfrac{1}{2} \cdot track \cdot \omega)/r$, $\omega_{right} = (v + \tfrac{1}{2} \cdot track \cdot \omega)/r$; same target for both wheels on a side	track 0.612 m, r = 0.085 m
Wheel-speed PI + feed-forward	`scout_amr_controller.m`	Torque per wheel from speed error and demanded acceleration	Kp 0.3, Ki 8, 17.5 N·m limit, Ts 5 ms
Slip management	`scout_amr_controller.m`	Slip ratio, traction window, torque limiting, friction estimate (section 10)	κ thresholds 0.12 / 0.08 / 0.40
Tyre-ground model	`scout_amr_tires.m`	Spring-damper normal force and smoothed Coulomb friction per wheel	kc 12 000 N/m, cc 700 N·s/m, v0 0.03 m/s
Collision monitor	`scout_wh_nav.m`	Overlap test of the robot circle against all obstacles; logs the clearance	r = 0.36 m

11.1 A* global planner

A* expands the cell with the lowest $f(n) = g(n) + h(n)$, where g is the cost so far and h the straight-line distance to the goal, which never overestimates the remaining cost. A move costs 1 for a straight step and $\sqrt{2}$ for a diagonal step. A binary heap holds the open set. After the search, line-of-sight shortcuts remove unnecessary waypoints, so the path becomes a few straight segments (the grey paths in Fig. 11).

11.2 Dynamic obstacles

Moving obstacles are handled in a reactive way. Every LiDAR scan marks the hit cells in the dynamic layer for 2 s, and the planner re-plans every 0.5 s or at once when the current path crosses an occupied cell. The motion of a moving obstacle is not predicted. If the path is blocked and an obstacle is closing in, the robot stops or backs away before it re-plans, because the stop-before-contact limit never allows a speed from which the robot could not stop in front of the nearest return.

12. System Architecture

Fig. 17 shows how the pieces connect. The Simscape Multibody plant contains the robot, the warehouse and the movers. The sensors deliver pose, speed and wheel speeds; the world model and the LiDAR deliver the obstacles; the autonomy block plans and decides; the drive control and the slip management turn the commands into wheel torques; and the tyre-ground model turns the wheel states into forces on the wheels. The pose is the true simulator pose, so the loop has no localisation block.

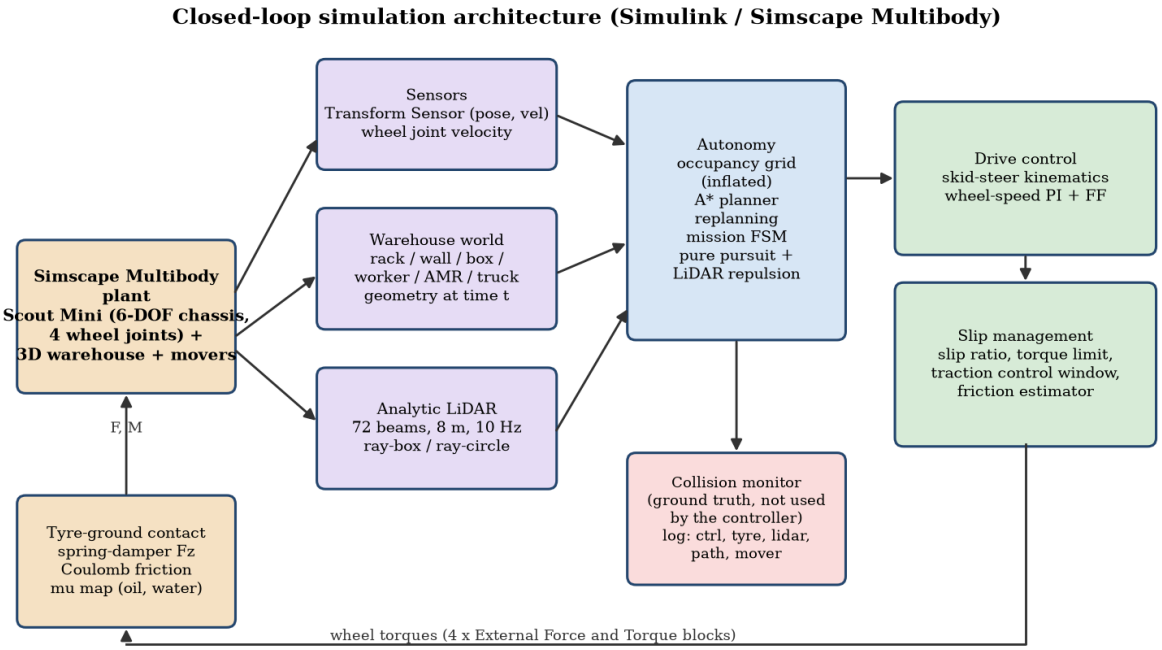


Fig. 17: Closed-loop architecture of the Simulink / Simscape Multibody simulation

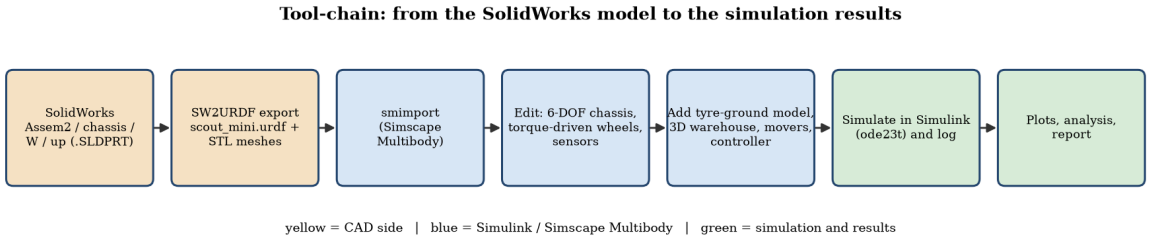


Fig. 18: From the SolidWorks model to the simulation results

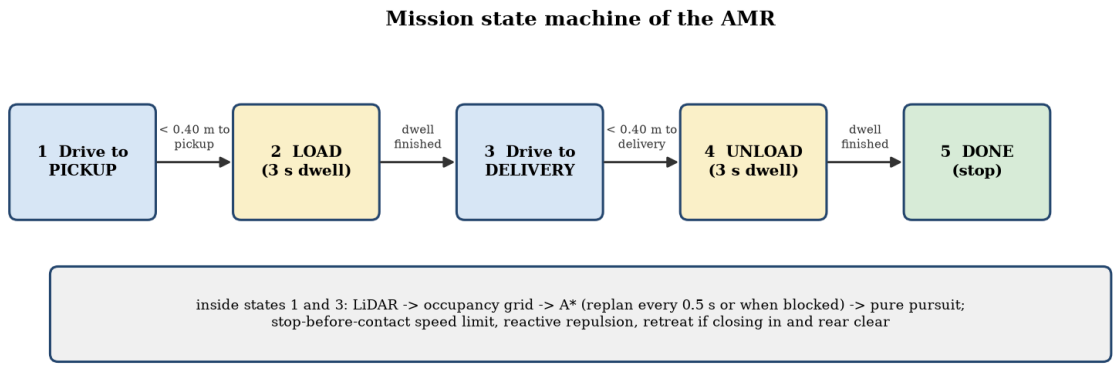


Fig. 19: Mission state machine

13. What Happens When the Code Runs

`run_scout_warehouse_simulation.m` loads the model, sets `tc_on`, simulates, extracts the logs, draws the figures and prints the summary. Inside the simulation, the controller block runs every 5 ms as shown in Fig. 20.

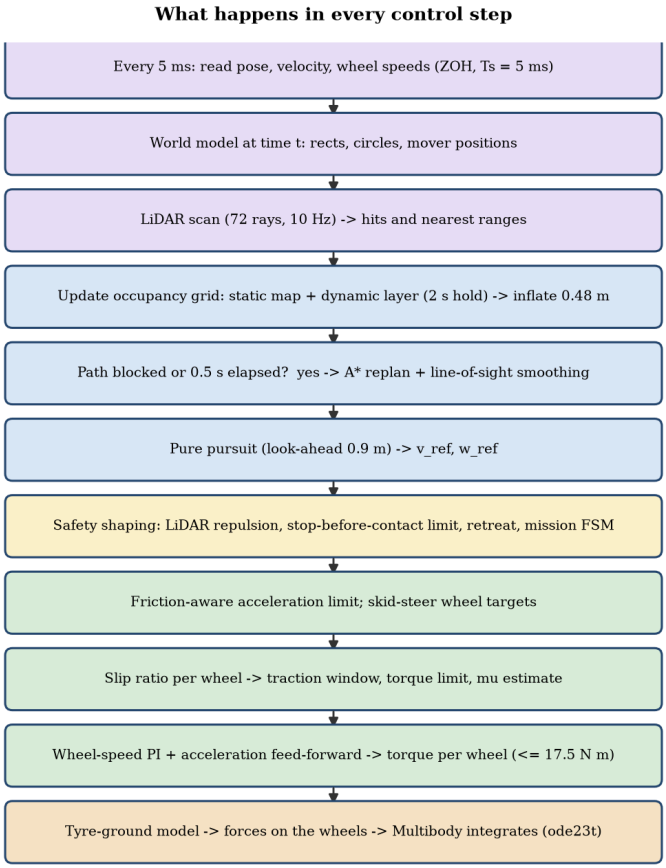


Fig. 20: Flow of one control step

- Setup: the model is loaded, the stop time is set to 120 s, and `tc_on` is set to 1 (or 0 for the comparison). The Mechanics Explorer shows the 3D scene when requested.
- Every 5 ms: pose, velocity and wheel speeds are sampled; the world is evaluated at the current time (the truck is released when the robot passes x = 12 m); the LiDAR is scanned at 10 Hz; the occupancy grid is updated.
- Every 0.5 s (or at once when the path is blocked): A* is run again from the current pose to the current goal.
- Control: pure pursuit gives the speed and yaw-rate references; they are shaped by the repulsion, the stop-before-contact limit and the friction-aware acceleration limit, converted into wheel targets, and the wheel-speed PI with slip management gives the four wheel torques.
- Physics: the tyre-ground model computes the forces, Simscape Multibody integrates the multibody equations with `ode23t` and moves the robot; the movers follow their scripted motion.
- End: when the mission state is done, the simulation stops itself; the logs are converted into the figures of this report.

13.1 Presentation outputs of the Simscape run

`make_example_style_outputs.m` turns the saved results of the reference run into the three presentation outputs of the project: a top-down trajectory map, a signal figure and an animation (amr_warehouse_avoidance.gif and .mp4 in the folder results). The map (Fig. 21) shows the driven path, the robot footprint every 2.5 s, the LiDAR fans, the racks, boxes and low-friction patches and the points where the route was corrected. The signal figure (Fig. 22) shows the clearance to the nearest obstacle, the controller commands, the wheel-speed commands and the chassis height and tilt. The chassis stays within 0.7 mm of the floor height and within 0.6° of tilt, which confirms that the contact model carries the robot without floating or sinking.

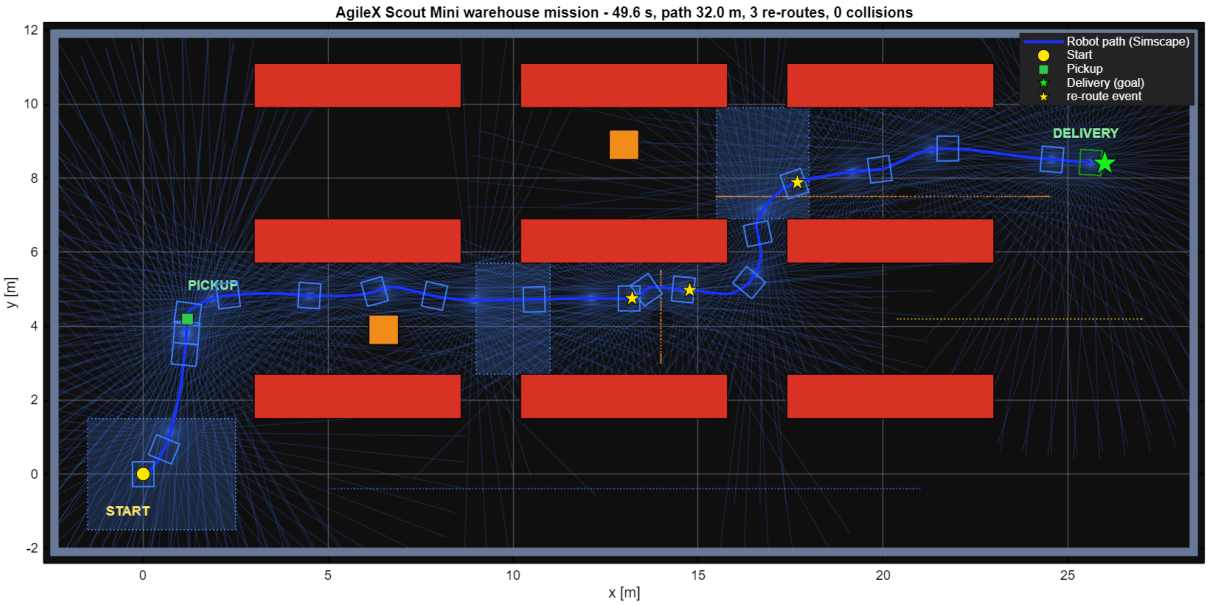


Fig. 21: Trajectory map of the Simscape warehouse mission (start, pickup, delivery, footprints along the path, LiDAR fans, re-route events)

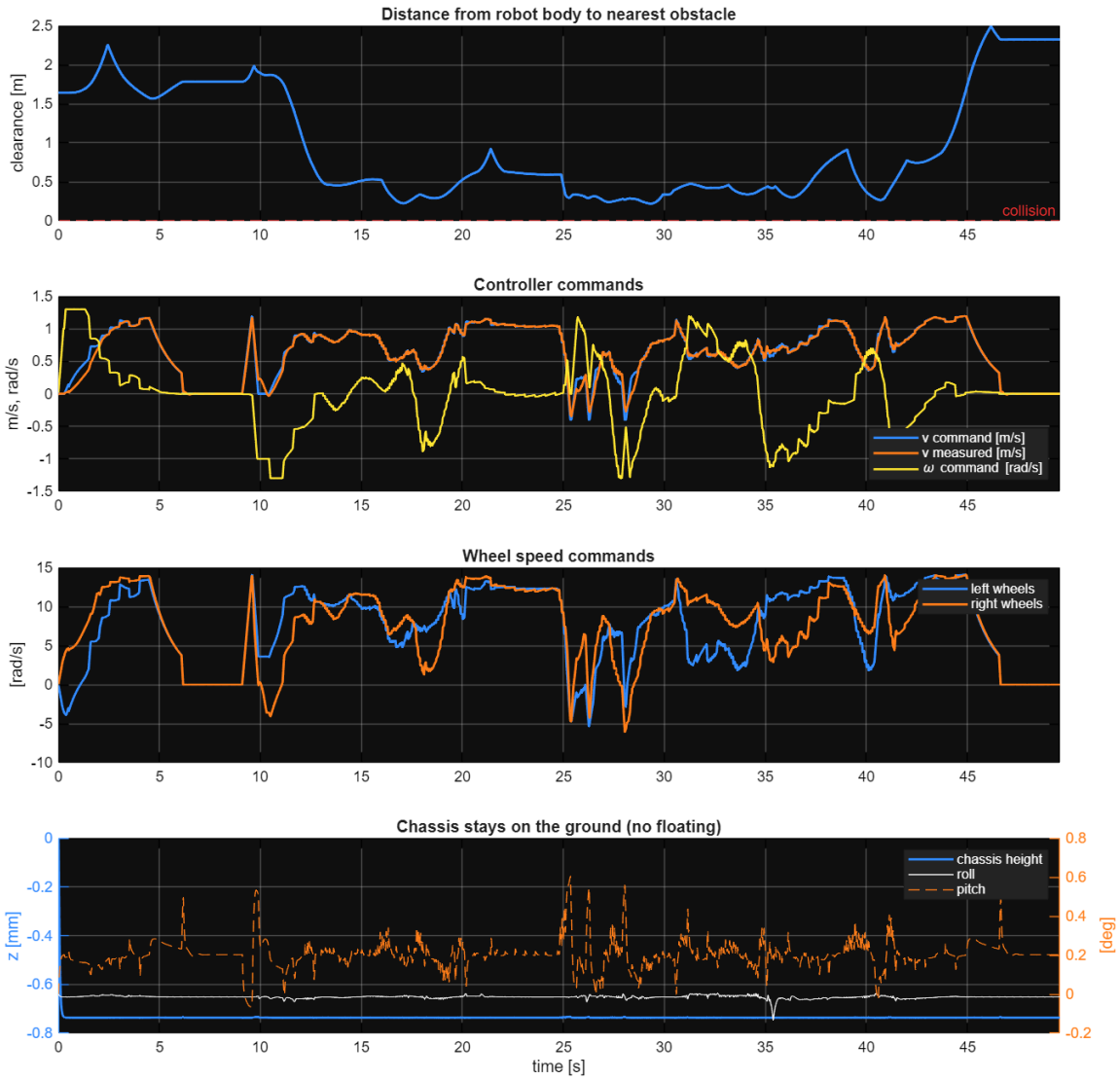


Fig. 22: Clearance, controller commands, wheel-speed commands and chassis height / tilt of the Simscape run

13.2 Mechanics Explorer view of the Simscape model

When scout_amr_sim is run (or updated) in Simulink, the Multibody Explorer opens with the full 3D scene of the Simscape model. Its model tree shows the four parts of the model (Moving_Obstacles, Scout_Mini_Robot, Warehouse, MechanismConfiguration) and the World frame, and the playback bar runs over the simulated 49.58 s of the mission. Fig. 23 shows the scene at t = 0 in the isometric view with the racks and goods, the blue low-friction patches, the pickup and delivery pads, the floor boxes and the moving obstacles. Fig. 24 is the top view at t = 3.6 s, when the Scout Mini has left the start dock towards the pickup station, and Fig. 25 shows the isometric view at t = 18.7 s, with the robot already in aisle 2 and the pallet truck at the east dock. The pictures are screenshots of the Multibody Explorer window.

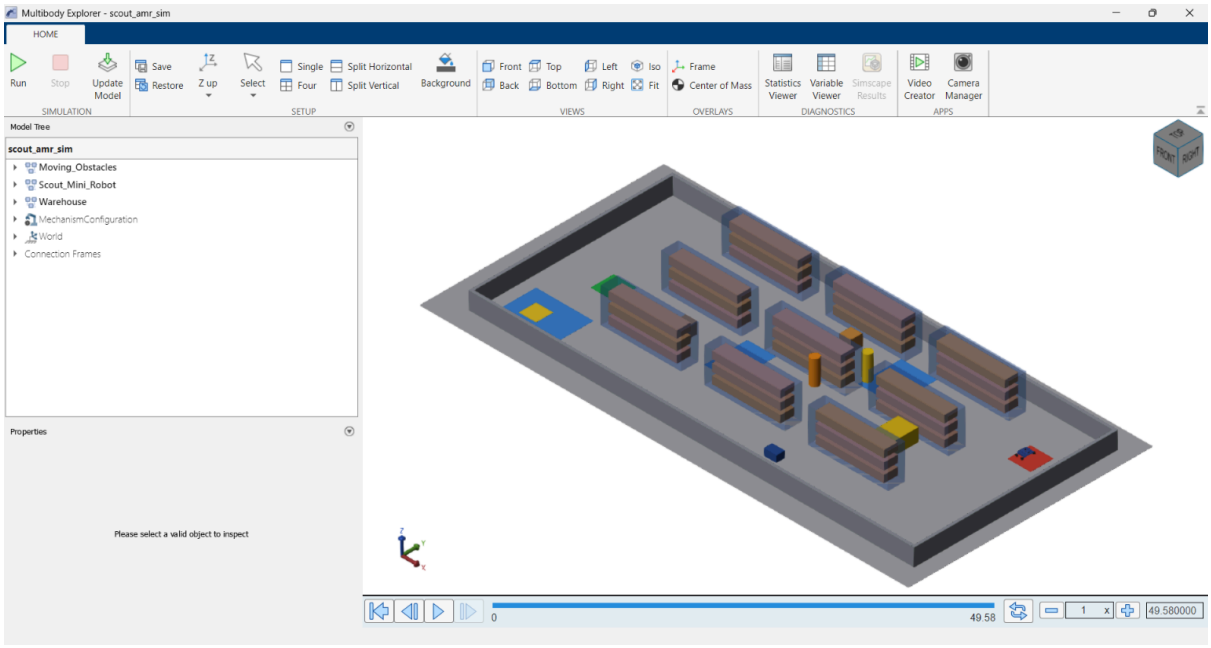


Fig. 23: Multibody Explorer of scout_amr_sim, isometric view at t = 0 (Scout Mini at the start dock)

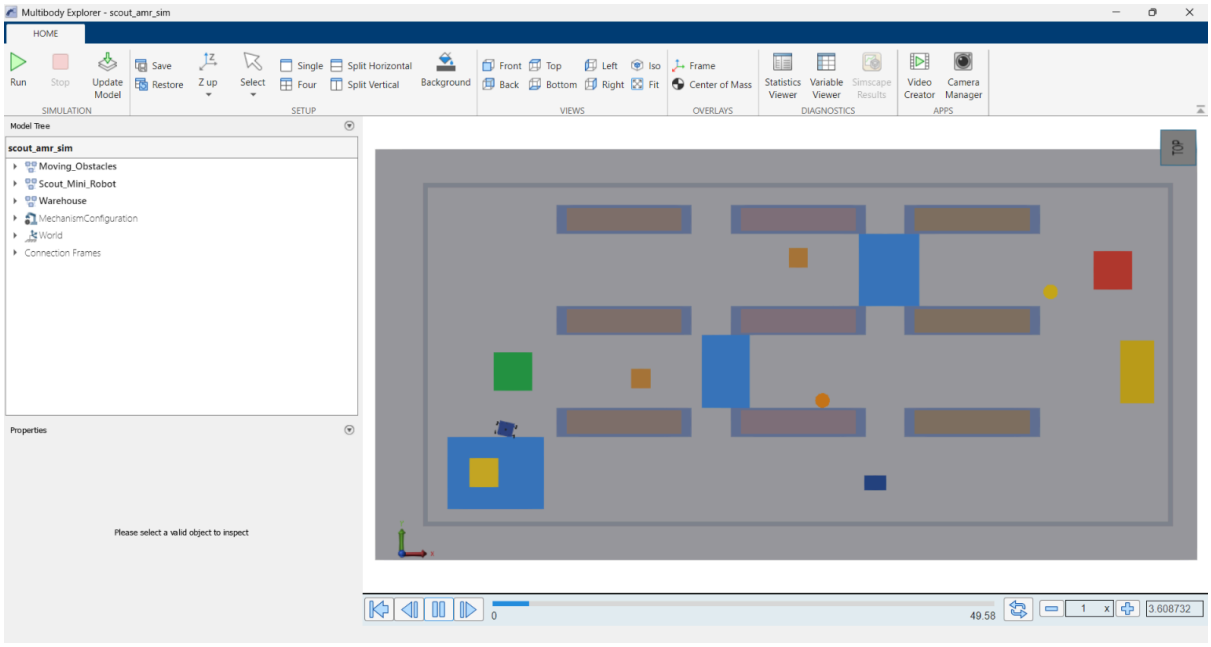


Fig. 24: Multibody Explorer, top view at t = 3.6 s (robot leaving the start dock)

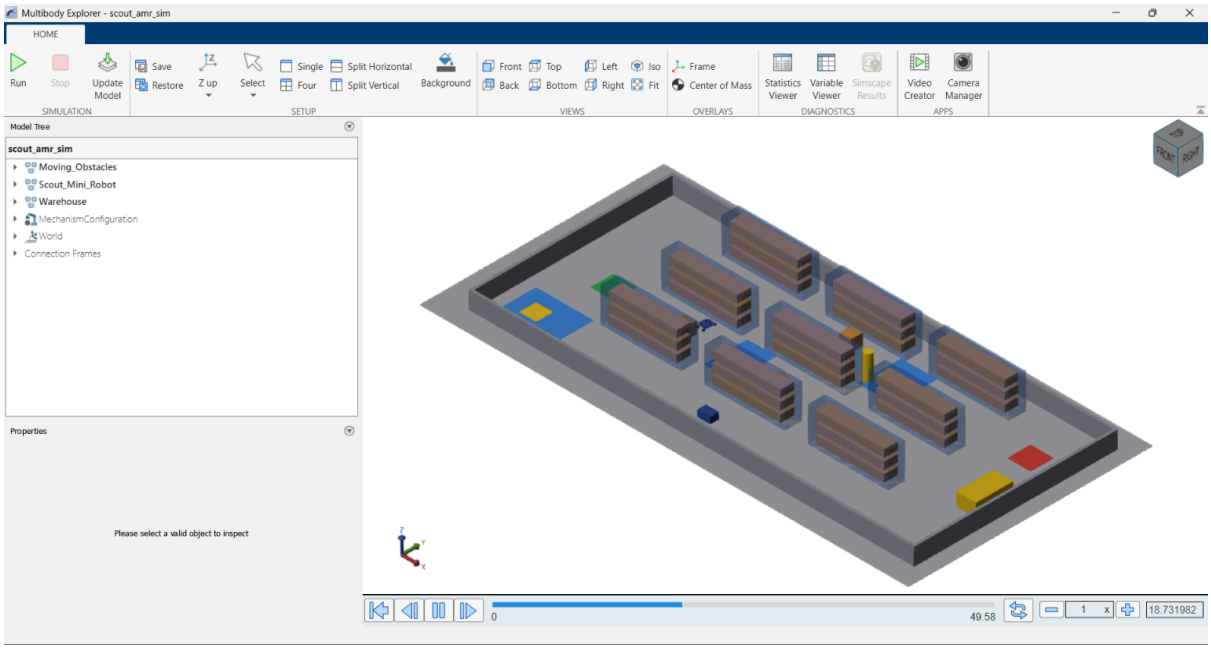


Fig. 25: Multibody Explorer, isometric view at t = 18.7 s (robot in aisle 2)

14. Additional MATLAB Visual Simulation (ISO 3691-4 Style)

Before the Simulink model, a MATLAB-script simulation of the same robot was written: `simulate_industry_amr_warehouse.m` (with `plan_warehouse_roadmap.m` for the route and `teleop_industry_amr_warehouse.m` for manual driving). It draws the robot with the same STL meshes in a 3D warehouse and shows how an industrial AMR behaves with safety fields. It is a kinematic and sensor-level model: the robot motion is integrated by the script, not by a multibody solver, and wheel slip is a drift model, not a contact simulation. Wheel-spin direction in the animation was corrected during this work.

Item	Configuration
Warehouse	26 × 16 m, 5 m ceiling; six three-tier racks in two rows (7 × 2 m, 3.6 m high); four static pallets; two extra hazards (crate in the west corridor, pallet in the central cross-aisle); outer walls
Loading dock	Ramp 3.5 × 3 m rising to a dock at 0.45 m; ramp friction 0.78, floor friction 0.68; depot (1.8, 2.0) m, dock (24, 13.5) m, two charging pads
Robot	30 kg + 12 kg payload, chassis 0.62 × 0.55 m, wheel radius 0.08 m, track 0.612 m, $v \leq 0.8$ m/s, $\omega \leq 1.4$ rad/s, acceleration 0.85 m/s², braking 1.8 m/s², hub-motor torque 17.5 N·m
LiDAR	64 beams, 8 m, $\sigma = 0.015$ m, 1 % dropouts, mounted 0.28 m above the chassis base
IMU and odometry	Gyro bias drift 0.0025 rad/s, noise $\sigma = 0.012$; wheel odometry with 2 % systematic drift (shows why LiDAR-based localisation is needed)
Safety fields (ISO 3691-4 style)	Warning 1.80 m (speed derating), braking 1.05 m (path bypass), protective stop 0.65 m (emergency halt)
Planner	`plan_warehouse_roadmap.m`: waypoint roadmap around known and extra hazards; re-routing counter, cool-down time

The script was run unchanged and its window was captured while it ran. The four screenshots in Fig. 26 to Fig. 29 show the isometric and the top (plan) camera at the start and later in the mission; the right-hand side of every screenshot is the live telemetry of the script (occupancy

grid, kinematics, safety clearance). In this run the robot reached the elevated dock after 44.7 s over 32.3 m with the path shown in Fig. 31.

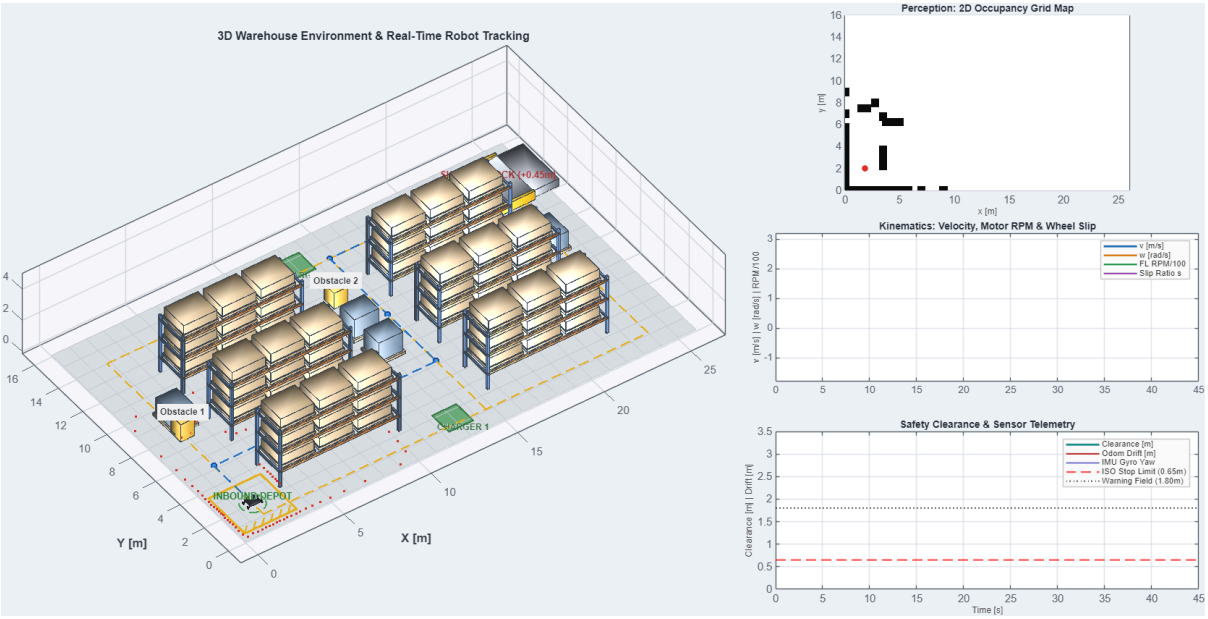


Fig. 26: Isometric camera at the start of the run (robot at the inbound depot)

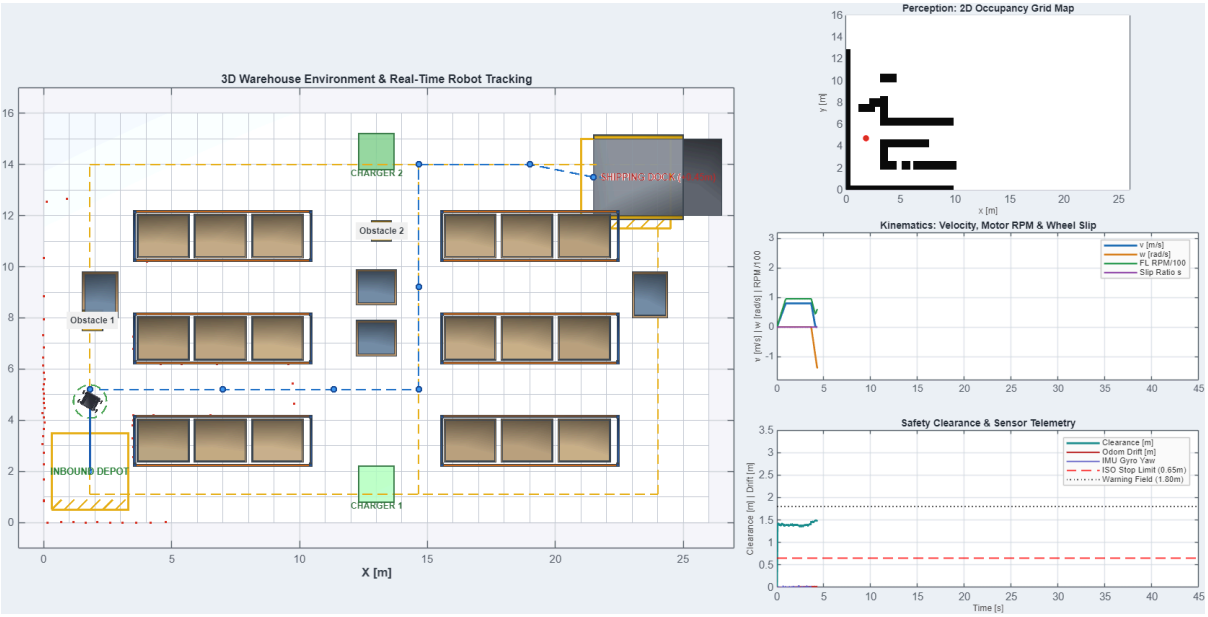


Fig. 27: Top (plan) camera: robot, waypoint roadmap, racks, pallets, charging pads and the shipping dock

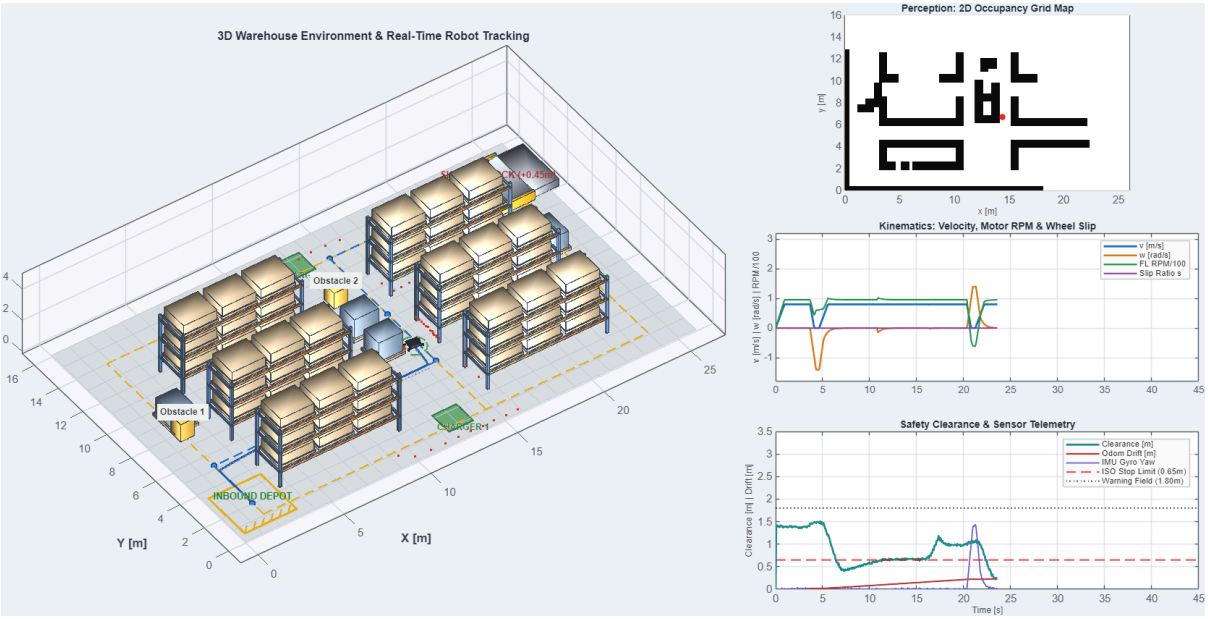


Fig. 28: Isometric camera later in the run (occupancy grid filling up, telemetry traces growing)

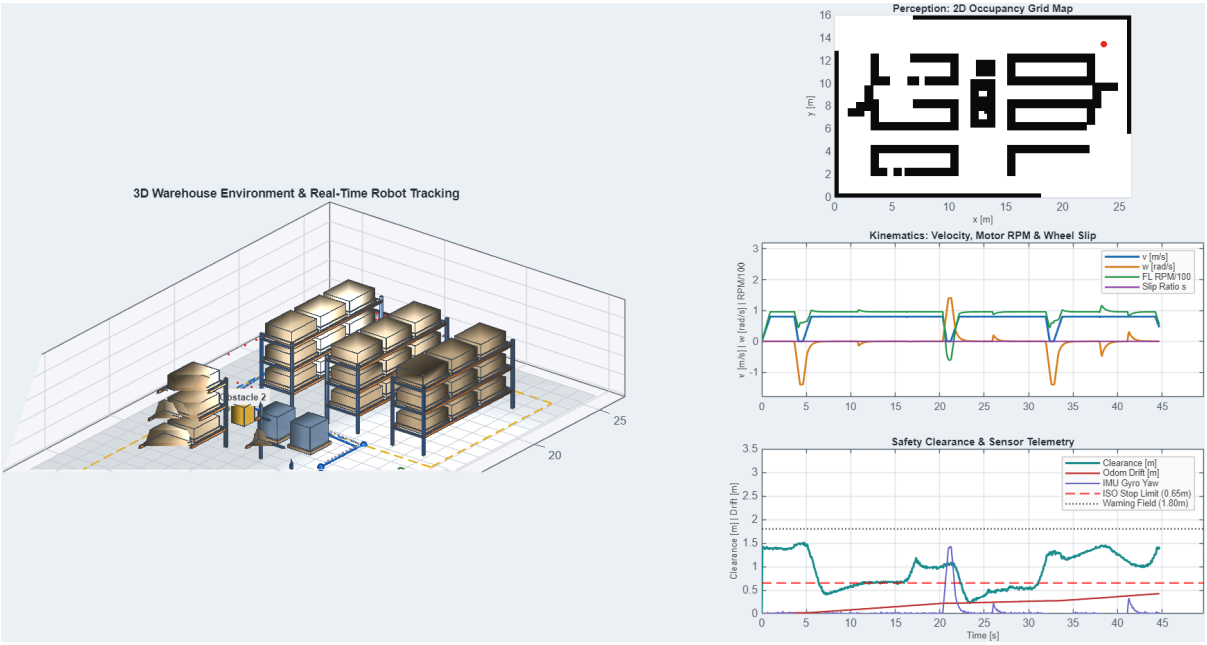


Fig. 29: Final state of the run: mission complete, telemetry over the whole 44.7 s

The telemetry of the run was extracted from the plots of the script and redrawn as separate graphs. Fig. 30 shows the linear speed (limit 0.8 m/s), the yaw rate (limit 1.4 rad/s) and the motor speed with the wheel slip ratio. The robot stops and pivots at the three corners of the route; the slip ratio stays at zero because the script has a drift model, not a tyre-ground contact simulation (this is the difference to the Simscape model of sections 7 to 10).

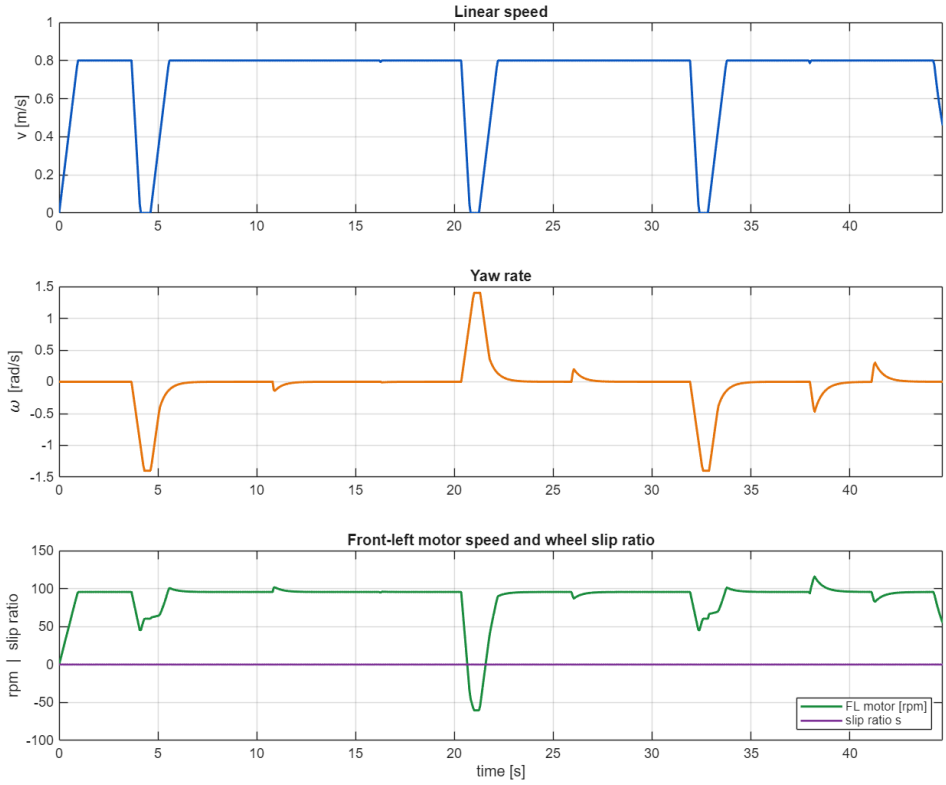


Fig. 30: Linear speed, yaw rate, front-left motor speed and slip ratio of simulate_industry_amr_warehouse.m

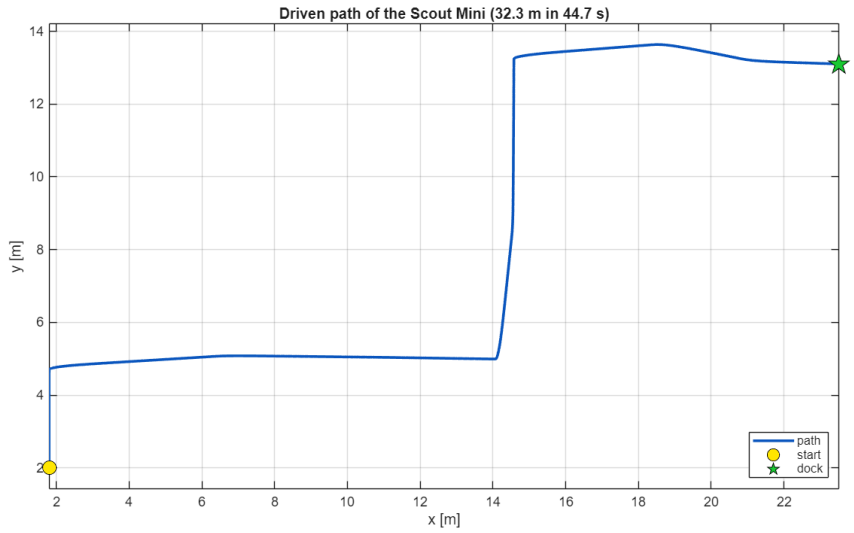


Fig. 31: Path driven by the script from the inbound depot to the shipping dock

Fig. 32 shows the clearance measured by the synthetic LiDAR against the three ISO 3691-4 style fields of the script (warning 1.80 m, braking 1.05 m, protective stop 0.65 m). The clearance falls below the braking field while the robot passes between the racks (6 s to 16 s and 22 s to 31 s) and reaches a minimum of 0.22 m at about 23 s; the script reports no collision. The clearance is the LiDAR range to the nearest hit, so it is not a body-to-obstacle distance and the values below 0.65 m show that the protective-stop field is not a hard limit in this script. The three large yaw-rate peaks of Fig. 30 and Fig. 32 are the turns at the route

corners. The odometry drift grows to 0.42 m because of the 2 % wheel-slip drift that was built in, which is why a real robot needs LiDAR-based localisation.

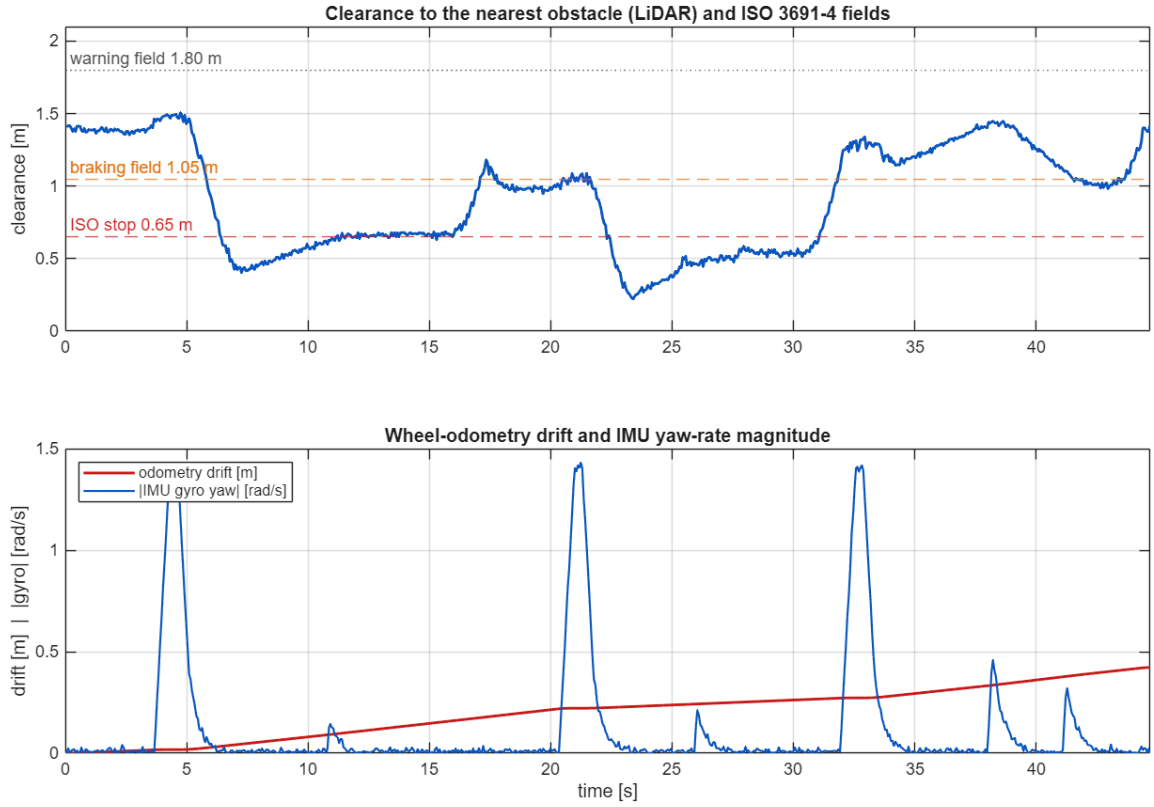


Fig. 32: LiDAR clearance with the ISO 3691-4 fields, odometry drift and IMU yaw-rate magnitude

15. How to Set It Up

Requirements: MATLAB R2025b with Simulink, Simscape and Simscape Multibody (the model uses the sm_lib and nesl_utility libraries); the Robotics System Toolbox is used for `importrobot` and the report figures. Nothing else needs to be installed.

- Copy the folder simulink_amr (it contains `scout_mini.urdf` and the meshes folder) to the computer.
- Open MATLAB, change to the folder and run `run_scout_warehouse_simulation`. The model `scout_amr_sim.slx` is built automatically if it is missing.
- To look at the model, run open_system('scout_amr_sim') and press Run; the Mechanics Explorer shows the warehouse.
- To switch slip management off, open the subsystem Mission_and_Slip_Controller and set the constant `tc_on` to 0, or use `run_scout_warehouse_simulation(false, true)`.

File	Purpose
`scout_mini.urdf`, `meshes/`	URDF of the robot and the STL meshes (base_link, w1…w4)

build_scout_amr_model.m	Imports the URDF (smimport) and builds scout_amr_sim.slx step by step
scout_amr_sim.slx	The Simulink / Simscape Multibody simulation model
scout_amr_params.m	All parameters: robot, tyre, friction patches, warehouse layout, LiDAR, grid, controller
scout_amr_controller.m	Mission navigation call, skid-steer control, slip management, logging vector
scout_amr_tires.m	Tyre-ground contact model (per wheel)
scout_wh_world.m	Warehouse geometry and moving obstacles at time t
scout_wh_lidar.m, scout_wh_plan.m, scout_wh_nav.m	LiDAR, A* planner, perception / planning / mission state machine
run_scout_warehouse_simulation.m	Runs the simulation, draws all figures, prints the summary
make_example_style_outputs.m	Trajectory map, signal figure and animation (GIF / MP4) of the Simscape run
results/	Figures 1–6, trajectory_map, signals, GIF / MP4 and the saved results (scout_warehouse_results.mat)
report/	Scripts that made the figures and this report
../simulate_industry_amr_warehouse.m and related	MATLAB-script visual simulation (section 14); report/capture_industry_full.m and make_industry_graphs.m capture its screenshots and graphs

16. Limitations and Future Enhancements

16.1 Limitations

- The pose comes from the simulator: there is no localisation, SLAM or sensor fusion, and the IMU and camera are only mounted, not simulated, in the Simulink model.
- The LiDAR is analytic and noise-free; collision detection is analytic. There is no Simscape contact between the robot and obstacles.
- The moving obstacles follow scripted paths; they are partly aware of the robot.
- The tyre model is a simple point-contact model per wheel; it has no tyre deformation, rolling resistance model or suspension.
- Motors are ideal torque sources limited to 17.5 N·m; battery, motor electrical dynamics and communication delays are not modelled.
- Torque and normal-load signals are noisy, and the slip comparison is one run per case.
- No validation against the real Scout Mini was done.

16.2 Future enhancements

- Add noisy LiDAR, IMU and wheel odometry as Simulink signals and replace the true pose by an estimator (EKF or AMCL / SLAM).
- Add contact forces between the robot and the obstacles (Simscape contact or Spatial Contact Force) so that collisions are physical.

- Model the motors and the battery with Simscape Electrical, and the drive line with Simscape Driveline.
- Predict the motion of workers and other vehicles instead of reacting to each scan; add fleet traffic rules at the cross-aisles.
- Test more layouts, random obstacle placements and several runs per slip configuration; tune the controller to reduce torque chatter.
- Validate the model against measurements of the real Scout Mini (mass, wheel radius, speed limits, slip on real floors).

17. Conclusion

The AgileX Scout Mini was modelled in SolidWorks, exported to URDF and STL, imported into MATLAB with `importrobot` and into Simulink with `smimport`, and turned into a dynamic Simscape Multibody plant with a free chassis, torque-driven wheels and a custom tyre-ground friction model. A 3D warehouse with racks, goods, stations, spills, static boxes and four moving obstacles was built into the same multibody scene. The robot navigates autonomously with a LiDAR, an occupancy grid, A* planning and re-planning, pure pursuit and a mission state machine, while a slip-management layer limits wheel slip. In the reference run it delivered the load in 49.6 s with 3 route corrections and 0 collisions, and it handled a floor box, a crossing worker, a blocked aisle and a spill-covered floor. With slip management the peak slip ratio was 0.48 instead of 0.64, at the cost of about 4.1 s of mission time.

These results come from simulation with the true pose, analytic sensing and scripted moving obstacles on a single invented layout, so they show that the software chain works, not that the physical robot would behave the same. The next steps are localisation with noisy sensors, physical contact with obstacles, a motor and battery model, and hardware tests.